Accepted Manuscript

A machine learning based approach to detect malicious android apps using discriminant system calls

Vinod P., Akka Zemmari, Mauro Conti

PII: S0167-739X(18)30621-6
DOI: https://doi.org/10.1016/j.future.2018.11.021
Reference: FUTURE 4585

To appear in: *Future Generation Computer Systems*

Received date : 20 March 2018
Revised date : 31 August 2018
Accepted date : 12 November 2018

Please cite this article as: Vinod P., A. Zemmari and M. Conti, A machine learning based approach to detect malicious android apps using discriminant system calls, *Future Generation Computer Systems* (2018), https://doi.org/10.1016/j.future.2018.11.021

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting proof before it is published in its final form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

ACCEPTED MANUSCRIPT

A Machine Learning based Approach to Detect Malicious Android Apps using Discriminant System Calls

Vinod P.[a], Akka Zemmari[b], Mauro Conti[c]

[a]*Department of Mathematics, University of Padua, Italy.*
[b]*LaBRI, Univ. Bordeaux, CNRS, France.*
[c]*Department of Mathematics, University of Padua, Italy.*

Abstract

The openness of Android framework and the enhancement of users trust have gained the attention of malware writers. The momentum of downloaded applications (app for short) from numerous app stores has stimulated the proliferation of mobile malware. Now the threat is the trend towards sophistication in malware being written to bypass signature-based detectors. In this paper, we investigate system calls to tackle mobile malware on Android operating system. To do so, we first employed machine learning to extract system calls. We then performed the empirical estimation of system calls derived from dataset employing human interaction and random inputs (generated with Android Monkey). After accomplishing intensive experiments on synthesized system calls with two feature selection approach, namely Absolute Difference of Weighted System Calls (ADWSC) and Ranked System Calls using Large Population Test (RSLPT), we validated the results on five datasets. All classifiers generated in Area Under Curve of 1.0 with an accuracy exceeding 99.9% suggest the appropriateness and efficacy of the proposed approach. Finally, we evaluated the effectiveness of classifier against adversarial attacks and found that the classifier is vulnerable to data poisoning and label flipping attacks. Adversarial examples created by poisoning malware samples resulted in the significant drop of classifier performance on perturbing 12–18 prominent attributes. Moreover, we implemented class label poisoning attacks which brought down the classification accuracy by 50% on altering labels of 50 malicious training instances.

Keywords: Malware detection; machine learning; classifier; feature selection; adversarial attacks; data poisoning; adversarial machine learning

1. Introduction

The Gartner's report summarizes global sales of smartphones over a year. It reports the global sales of smartphones in the year 2017 as 380 million units with a 9.1% increase over first quarter of 2016. The plethora of Android applications and the openness of Android markets have added to the popularity of smartphones. Further, this has created an opportunity for the malware authors to sneak in and cause copious instances where malware is hidden behind benign applications threatening the privacy and security of users.

To protect the users from malware threats, security vendors offer different tools which are usually based on signature matching approach. Signature-based detection is prone to several challenges such as database with patterns of known threats, frequent update of the signature repository and intervention of experienced personnel in the signature generation process. In the present scenario, where malware attack is tremendously increasing, it is an extremely difficult for a pattern matching based scanners to detect new variants of existing malicious programs. As a result, there arises the need for devising alternate methods to detect malware.

Malware analysis approaches are broadly classified as static, dynamic and hybrid. Static analysis involves extracting features from malware after installing the app on a device or emulator. Such systems investigate the presence of

☆

Email address: pvinod21@gmail.com (Mauro Conti)

1

ACCEPTED MANUSCRIPT

known patterns, permissions, APIs and perform control flow analysis of an application without their execution. The approach is fast, however, unavailability of complete source code might not reveal malicious behavior. Anti-malware scanners employing static analysis can be evaded by transformation attacks involving code obfuscation and reflection. Moreover, control flow analysis involves computational overhead and is thus not suitable to be performed on the device. In dynamic analysis, malware samples are executed in an emulated environment such as a sandbox. Certain dynamic behavioural based anti-virus engine, observers the presence of sensitive APIs (such as read contacts, Internet, sending SMS), and assumes it as the malicious artifacts. Later, these malware detectors alert the user to suspicious behaviour. However, benign messaging, travel and tourism apps also make use of these APIs for legitimate reasons. Therefore, these systems may raise the false alarm and make them less predictable.

New malware samples evolve by employing code obfuscation, manifest cheating, and loading malicious code while in execution. Even if the fingerprint of original malware sample is available with the anti-malware engine, new derivatives of the parents hide from detection. Although these techniques can help malware to remain stealth, the runtime behaviour in-terms of OS interaction using system calls can be used reliably to identify them. Malware scanners using machine learning techniques constructed with robust features and classifiers are surely a solution to fight malware attacks. Machine learning algorithms are thus used to build classification models by training classifiers with prominent features mined from datasets. Consequently learned models are used to identify unseen samples. This paper utilizes the machine learning techniques accompanied by dynamic analysis to distinguish between malware and benign behaviors. The study was conducted on five datasets; here system calls are considered as features for analysis. System calls were selected as they represent the activities related to battery consumption, memory accesses, incoming and outgoing traffic and sensor status of a smartphone. Extraction of features was performed in two ways (a) using Android Monkey and also (b) by human interaction. Android Monkey is a utility which is a part of Android SDK and simulates human interaction with the application. Independent as well as a sequence of system calls, were derived from the extracted call logs.

The extracted features space containing noises do not contribute towards the detection process. Hence, an optimal feature vector must be derived. Generally, use of feature selection methods reduces the high-dimensional feature space by identifying subsets of qualitative attributes enhancing the performance of classifiers. This paper thus proposes two feature selection methods Absolute Difference of Weighted System Calls (ADWSC) and Ranked System Calls using Large Population Test (RSLPT) for pruning higher dimensional system call set. An optimized feature occurrence matrix with each element representing the Term Frequency and Inverse Document (TF–IDF) value is built and, classification is carried out using ensemble algorithms. Furthermore, we investigate the viability of malware scanners by subjecting it to adversarial examples. These instances are created by (a) poisoning malicious input vectors and (b) altering the class labels to mislead the classification algorithm. Evaluation also validates the robustness of the different solution and open the research community to develop secure classification model resistant to adversarial attacks. Through our work, we make following contributions.

- We demonstrated the effectiveness of feature selection in Android malware classification system. To our knowledge, most of the prior work in the domain of malware detection utilizes conventional approaches such as information gain, chi square attribute ranking, correlation-based filters, Fischer score etc. We focused predominantly on synthesizing relevant attributes for creating qualitative classification models fruitful for real-time analysis of malicious code. Hence, we implemented two new methods *Absolute difference of weighted system calls (ADWSC)* and *Ranked system calls using large population test (RSLPT)* that exhibited in an exceptional performance in the identification of malicious applications.

- With exhaustive experimentation considering variable feature length we showed that a sub-optimal feature space can be extracted that minimizes false positives. We also conclude that representation of feature vector plays an important role in classifier performance. Thus, comparative analysis of different categories of feature vectors was performed. Finally, we concluded that application vector where each element represented as TF–IDF weight excelled compared to their counterpart. By doing so diversity between feature vector was enhanced and subsequently resulted in higher prediction accuracies ranging from 99.81% to 100% with AUC of 1.0.

- We empirically show the significance of system calls based scanners in comparison to other attributes such as permissions and APIs. Also, we evaluate the performance of proposed feature selection methods with symmet-

ACCEPTED MANUSCRIPT

ric uncertainty, information gain and principal component analysis (PCA). We show the improved performance of proposed feature selection over conventional methods.

- We evaluate the effectiveness of classifier against adversarial attacks. More specifically to the best of our knowledge, we are the first to implement data poisoning and label flipping attack on the system call feature set. Also, none of the prior work except one (discussed in Section 2) considers attacks on the developed classification model. Moreover, we experimentally conclude that carefully created adversarial samples can easily evade detection, suggesting the development of security aware classification model.

The rest of the paper has been organized as follows. In Section 2 we introduce the prior work conducted by researchers and compare different approaches based on features, attribute selection methods, classifiers. Section 3 outlines the methodology that has been implemented. Section 4 presents the experiments and results. Attacks on machine learning based scanners are covered in Section 5. Also, we evaluate the performance of poisoning samples and class labels. Section 6 discuss the outcome of the study and validity of threats respective to our proposed approach. Comparative analysis is covered in Section 7. Finally, in Section 8 we conclude the paper with directions for future work.

2. Related Work

Android malware analysis using machine learning can be categorized into (a) static (b) dynamic and (c) hybrid approaches. In the following subsection we introduces the aforementioned methods.

2.1. Static Analysis

Canfora et al., [25] investigated the effectiveness of opcode *ngram* sequence for identifying Android malware. Relevant *ngram* were extracted by estimating the *relative difference* of *ngram* frequencies between malware and trusted applications. Experiments demonstrated an accuracy of 96.88% considering 1000 *bigrams*. Authors in [6] examined structural features and evaluated the performance classification model using classifier like Naïve Bayesian, Random forest, Logistic Regression and Support Vector Machine. Top 10 features selected using Information gain, and model learned with Random forest produced improved outcomes. Android malware characterization was performed in [7]. The classification was conducted on the basis of installation methods, activation mechanisms and peculiarities of malicious payloads. Further, DREBIN [5] combines concepts of static analysis and machine learning. Later, authors in [9] used manifest and opcode features for identifying malicious files. Two feature selection methods i.e., entropy–based category coverage difference and weighted mutual information were applied. Overall analysis exhibited that the learning models developed with weighted mutual information were observed to be superior compared to entropy-based category coverage difference. Authors in [10] proposed Multi-level Anomaly Detector for Android Malware (MADAM), and probabilistic discriminative model was investigated [13]. Comparative analysis of eleven known and newly discovered attributes were performed in [19]. New attributes included file entropy in the resource-folder, GUI layout, number of methods and instructions per method. Author in [15] proposed a machine learning based malware detector employing 48 attributes selected using mutual information. The classification model was developed with Bayesian classification approach. An overall accuracy of 97% was reported using 5–fold cross-validation. Extension to their work [16], authors proposed detection of the malicious application using parallel classification algorithms. The evaluation was ascertained using 53 significant attributes, finally 97% accuracy was reported. Wei et al. [44] utilized requested/used permissions, filtered intents, code-related information, restricted/suspicious API calls and hardware related information. An accuracy of 99.36% was obtained in the detection malicious apps and the best accuracy of 82.93% in the categorization of trusted apps. PIndroid [47] uses combination of permissions and intents alongwith ensemble methods for detection of malicious apps from collection of 1,745 applications. They used statistical and machine learning methods to validate their hypothesis. PIndroid is reported to provide 99.8% accuracy. Milosevic et al. [45] presented two machine learning approaches (i.e., classification and clustering) adopting permissions and bag-of-words. Accuracy of 95.8% was obtained with ensembles of classifiers. Grosse et al. [27], evaluated the performance of Android malware detector employing the neural network, learned over classes of binary features from manifest files. The goal was to classify malicious instances as benign. Hence, the adversarial examples were crafted based on the Jacobian matrix. To this end, 63% of samples were reported to be misclassified. Later in their

ACCEPTED MANUSCRIPT

study authors proposed two defense mechanisms for hardening malware detectors namely, defensive distillation and retraining. Distillation increased the performance by a marginal amount. However, retraining the classifier produced promising results by improving classification rate. Android-based malware detection designed by multi-feature collaborative decision fusion (MCDF) was proposed by authors in [40]. MCDF resulted in precision and recall in range of 98.31–98.91% with 135 permissions and 210 API calls. Further, DroidNet [41] employed permissions, sensitive API calls, system events and permission rate, from 2130 samples. An accuracy of 88.2% was achieved with Rotation forest. As Rotation forest are compute intensive, also comprehensive analysis on large dataset was not conducted, hence the applicability of DroidNet cannot be justified.

2.2. Dynamic Analysis

Canfora et al.[26], proposed detection using sequence of 750 system calls and achieved accuracy of 97% with call subsequence of length $n = 3$. Additionally, a machine learning based classification system employing Linux kernel based features was proposed in [28]. In this work, a total of 59 features divided into three categories (memory, CPU, and Network) were extracted. The study revealed a detection rate of 95.97% with the elimination of 23 unnecessary attributes. Further, to precisely capture malicious behaviour, authors in [8] simulated human interaction. The proposed system demonstrated an improved performance compared to other automatic analysis methods. Authors in [43] presented a detection framework using system calls extracted from 200 applications. During the filtering phase, irrelevant system calls were eliminated, subsequently, system calls with identical functions were consolidated. However, mapping multiple system calls with few representative ones cannot be considered feasible as the system calls have indistinct return types and parameters. Later, authors in [42], employed sequence of system call with variable depth for generating malware and normal patterns from a collection of 2000 samples. An accuracy exceeding 90% with 7.69% FPR was reported by optimizing set of patterns and threshold. Xi Xiao [39] et al. considered 196 system calls, and examples were trained on back propagation neural network. The highest $F-score$ of 0.982 at TPR of 0.977 and a FPR value of 0.013 was reported. Moreover, better classification outcome was ascertained with learning rate of 0.7. Authors in [29] developed a machine learning based application capable of detecting malicious apps. The analysis was performed using multiple classifiers, however in all cases, decision tree and REPTree resulted in better performance. Likewise, Andromaly a host-based malware detection system was proposed by authors in [12]. Chi-Square, Fisher Score, and Information Gain were used as feature selection methods to filter significant attributes. The proposed approach identified continuous attacks, however, learning model was trained on a limited number of examples. Authors in [14] captured the TCP packets, and 11 features were extracted from the packets. Using *ClassifierSubsetEval*, a feature selection method six out attributes were extracted. The experimental results indicated 99.99% accuracy. Finally, SWORD [49] identifies evasion-aware malicious apps using Asymptotic Equipartition Property (AEP). The property states that information corresponding to a graph is confined in few paths of the graphs. A detection accuracy of 94.2% on a dataset consisting of 2000 applications was obtained.

2.3. Hybrid Analysis

In the review paper [11], 100 articles were analyzed with the perspective of feature selection in Android malware detection. The features extracted from samples were categorized into four groups i.e., (a) static (b) dynamic (c) hybrid and (d) meta-data features. Besides, the authors reported that feature selection algorithms were not exhaustively investigated in prior articles. A dual defense mechanism for malicious apps was proposed by authors in [20]. The scanner contained two phases (a) in the first phase samples were executed in a sandbox environment and system calls were logged (b) later the network activity of each app was monitored. Wrongly labelled files were corrected if any application depicted suspicious network activity. Finally, the proposed framework with Random forest classifier resulted in 99.2% accuracy. Further, authors proposed a system known as *Manilyzer* [21], exploiting information present in Android-Manifest.xml file. Classification accuracy of 90% on an app corpus consisting 617 applications was reported. The detection of smartphone malware from 200 applications using the subset of system calls, the weighted sum of permissions and combination of permissions was addressed in [24]. Experimental study reported statistical difference in `open`, `read`, `recv` and `write` system calls, and claimed 85% precision. Authors in [31] proposed a three-phase detection and classification framework. The first phase known as Permission Based Detection (PBD) identifies samples by comparing the permission vectors of training and inspected programs. Applications reported as suspicious were passed to the second phase known as System–Call Based Detector (SBD). Later, the sequence of system calls

ACCEPTED MANUSCRIPT

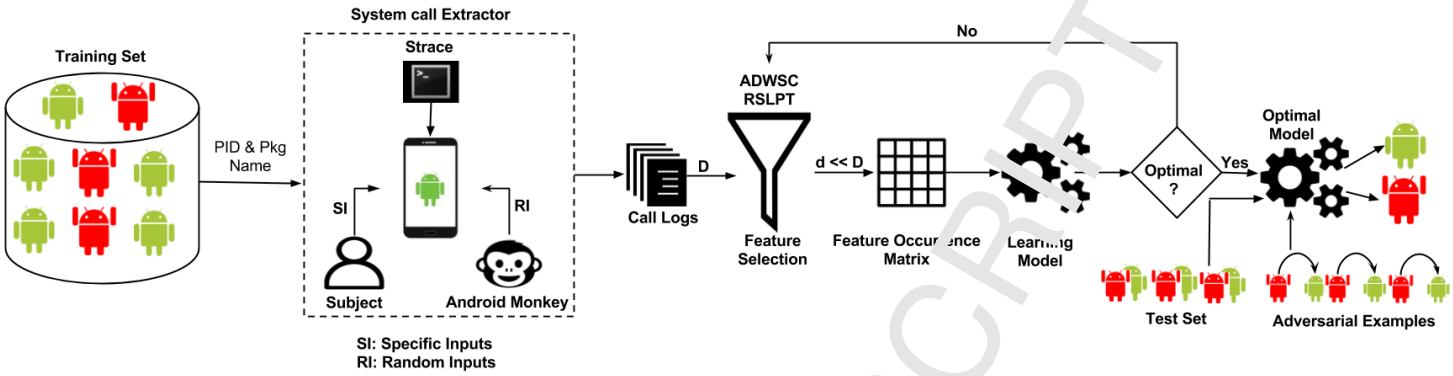


Figure 1: Proposed Methodology

were mapped to bit vectors, and the samples in the test set were flagged malicious if the computed cosine similarity of the bit vectors of train and test instances was reported higher than a fixed threshold. The study demonstrated a True Positive Rate(TPR) of 97% with False Positive Rate(FPR) of 3%. A 3-level hybrid framework known as SAMADroid combining (a) static and dynamic analysis (b) local and remote host, and (c) machine learning was proposed by authors in [48]. Experimental results showed that the Random forest provides 99.07% accuracy on static features, and 82.76% on dynamic features.

On contrary to above discussed research work, in this paper, we propose a solution for identifying malicious samples by deriving discriminant system calls using two feature selection methods named ADWSC and RSLPT. We compare the performance of the attribute set with conventional feature selection methods such as information gain and symmetric uncertainty. Also, we analyzed and compared our finding with principal component analysis. Later, we present that system call based android malware detectors can be misled by adversarial examples derived by carefully introduced perturbation. Moreover, we present that the performance of detectors substantially drops on poisoning the class labels

3. Methodology

In this section, we outline our proposed methodology. Following are the steps used in the identification of malicious samples, also refer to Figure 1:

- Dataset collection and cleaning (preprocessing).
- System call extraction with the help of automated tool and human interaction.
- Elimination of irrelevant attributes by employing feature selection algorithms.
- Construction of feature occurrence matrix using prominent attributes.
- Evaluation of classification model using cross-validation.

3.1. Feature Extraction

System calls logged during the execution of each sample are considered as the features for identifying files as malware or benign. The primary reason for recording system call is that it reflects the behavior of the executing program and its interaction with the operating system kernel. The system calls have been extracted by two mechanism (a) using an input generator i.e. Android Monkey which is used to perform stress testing on mobile applications (b) another method is running the application in an emulator and a subject (experienced user) interacted with the app.

Employing Android Monkey, 2500 events were triggered by an application and the and generated system calls were collected, additionally, a single human interacted with each application for five minutes. Each system call in

ACCEPTED MANUSCRIPT

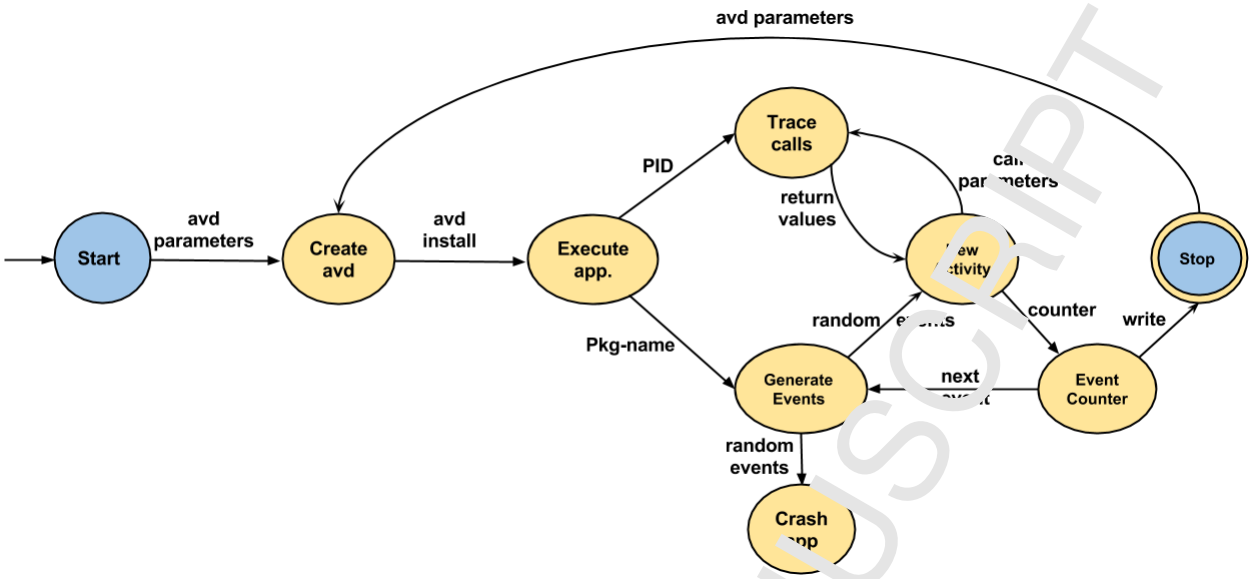


Figure 2: State transition diagram for feature extraction.

the trace consisted of system call name, parameters and return values. Initially, our study considered only call name as an attribute for discriminating malware and benign files. Further, performance with system call sequence is also exhaustively studied.

Figure 2 explains feature extraction phase. Initially, a clean Android virtual device is created by specifying basic configuration such as device name, memory size and certain parameters which resemble real device. Then .apk files are installed in an emulator using command `adb -s emulator-id install sample.apk`. Subsequently, each sample is executed in the emulator and `process id` (pid) along with `package names` are extracted. System calls are recorded (`strace -p <pid> -o sdcard/logfilename.txt`) and during this course, Monkey tool interacts with app providing random events (`adb shell monkey -p pkg-name v 2500`). This process continues until the specified events are exhausted or an incorrect application path is explored, which results in failure of its execution. Finally, the emulator is again restored to a clean state. System call extraction is introduced below:

- Install an application in the emulator. Fetch zygote process id (PID)-using *ps* command, and execute `strace` on zygote.
- Run the application for 60 second, extract the process id of the application.
- Interact with the application using Android Monkey for fixed time interval, using application packagename.
- Simulate events corresponding to phone, SMS and movement one at a time each for 30 seconds. Finally, restore the virtual environment to safe state.

3.2. Feature Selection

Feature selection techniques aim at a larger understanding of the data, minimizes computations involved during training and testing phase thereby improving the predictor performance. It also generalizes the learned models and thus reduces over fitting. Formally feature selection algorithms are broadly classified into two approaches (a) *feature search* and (b) *feature subset evaluation*.

One of the techniques that have been widely used in exploring attribute space is *feature search*. Two types of feature search proposed in machine learning domain are classified as (a) *forward selection* and (b) *backward elimination*. Forward selection starts with an empty set and incrementally adds one feature at a time. However, backward elimination, begins with the complete set of attributes and eliminates insignificant features one by one. Feature subset evaluation is an alternative technique that is widely used to identify irrelevant and redundant attributes. There are

ACCEPTED MANUSCRIPT

three categories of feature evaluation techniques (a) *filter* (b) *wrapper* and (c) *embedded* methods [30]. In the present study two feature selection methods based on filter approach are utilized to determine optimal feature vector. Table 1 lists frequently used notations in this paper.

Table 1: List of notations used in the paper.

Symbol	Meaning
s_i	system call
M	set of malicious apps
B	set of benign apps
$Occ(s_i,S)$	total occurrence of s_i in set S
$Occ(s_i)$	total occurrence of s_i
$T(s_i,M)$	ratio of a system call s_i in the malware dataset divided by sum of all system calls
$T(s_i,B)$	ratio of a system call s_i in the benign dataset divided by sum of all system calls
$diff_{s_i}$	absolute difference of mean values of s_i in the target classes
$StdErr(\overline{x_B}-\overline{x_M})$	standard error
ϕ_{s_i}	ratio of difference ($diff_{s_i}$) divided by standard error
$n(d,s_i)$	number of instances (d) invoking system call s_i
$D(s_i,M)$	logarithm of sum of total number of malware and benign files divided by frequency of a system call s_i in the malware apps.
$D(s_i,B)$	denotes the logarithm of sum of total number of malware and benign files divided by frequency of a system call s_i in the benign apps.

Following subsections introduce feature selection methods employed in the experiments.

3.2.1. *Absolute Difference of Weighted System Calls (ADWSC)*

ADWSC is used to weigh each system call and assign importance based on the number of times it is invoked by the application. In this approach, the basic idea is to decrease the weight of system calls that uniformly exist in the target classes, although increasing the weight of system call that are specific/prominent to a class. The weighted absolute difference of a system call is determined using equations through (1) to (7).

$$T(s_i,M) = \frac{Occ(s_i,M)}{\sum_{j=1}^{|M|} Occ(s_j)}, \tag{1}$$

$$T(s_i,B) = \frac{Occ(s_i,B)}{\sum_{j=1}^{|B|} Occ(s_j)}, \tag{2}$$

$$D(s_i,M) = \log\left(\frac{|M|+|B|}{\{n(d,s_i)|d \in M\}}\right), \tag{3}$$

$$D(s_i,B) = \log\left(\frac{|M|+|B|}{\{n(d,s_i)|d \in B\}}\right), \tag{4}$$

ACCEPTED MANUSCRIPT

$$Wt_M(s_i) = T(s_i, M) * D(s_i, M), \tag{5}$$

$$Wt_B(s_i) = T(s_i, B) * D(s_i, B), \tag{6}$$

$$d(s_i) = abs(Wt_M(s_i) - Wt_B(s_i)). \tag{7}$$

M and B in the above equations denote malware and benign classes respectively and s_i represents a system call in the dataset. Equations 1 and 2 are used to calculate the number of invocations of the system calls in malware/benign applications. In particular, the aforesaid equations identifies how frequently system call occurs in call logs. $D(s_i, M)$ denotes the logarithm of the sum of the total number of malware and benign files divided by the frequency of a system call s_i in the malware apps. Similarly $D(s_i, B)$ for the benign dataset is determined. Using equations 3 and 4 the weights of frequently accessed calls are decreased, although the weight of attributes specific to a class is increased. $Wt_M(s_i, M)$ denotes the product of $T(s_i, M)$ and $D(s_i, M)$, likewise, we also calculate $Wt_B(s_i)$. Both $Wt_M(s_i, M)$ and $Wt_B(s_i)$ denote the score/weight of each system call. Equations 5 and 6 assigns higher weights to calls that are frequently invoked by a particular class, specifically such attributes carry high discriminating power. Besides, system calls that occur infrequently in a class and appear virtually in all instances will receive lower weights Finally, the absolute difference (see Equation 7) of the obtained weights of system calls is ascertained and eventually arranged in descending order of weights. We present ADWSC feature selection approach in Algorithm 1.

Procedure 1 Weighted System call extracted using ADWSC

procedure ADWSC(S) ▷ $S = \{s_1, s_2, \cdots, s_N\}$; set of system calls.
  while $i \neq |S|$ **do**
    ▷ **Determine weight of system calls in malware set.**
    $sum_n \leftarrow sum_n + Occ(s_i)$
    while $j \neq |M|$ **do**
      if $s_i \in d_j$ **then**
        $sum_d \leftarrow sum_d + Occ(s_i, d_j)$
        $dcount \leftarrow dcount + 1$
        if $sum_n > 0$ **then**
          $T(s_i) \leftarrow \frac{sum_d}{sum_n}$
          $D(s_i) \leftarrow \log\left(\frac{|M|+|B|}{dcount}\right)$
          $wt(s_i, M) \leftarrow T(s_i) \times D(s_i)$
    ▷ **Determine weight of system calls in benign set.**
    $sum_n \leftarrow 0$
    $sum_d \leftarrow 0$
    $dcount \leftarrow 0$
    while $j \neq |B|$ **do**
      if $s_i \varepsilon d_j$ **then**
        $sum_n \leftarrow sum_n + Occ(s_i, d_j)$
        $dcount \leftarrow dcount + 1$
        if $sum_n > 0$ **then**
          $T(s_i) \leftarrow \frac{sum_d}{sum_n}$
          $D(s_i) \leftarrow \log\left(\frac{|M|+|B|}{dcount}\right)$
          $wt(s_i, B) \leftarrow T(s_i) \times D(s_i)$
    $d(s_i) \leftarrow |\, wt(s_i, M) - wt(s_i, B)\,|$
    return s_i ▷ weight of a system call

ACCEPTED MANUSCRIPT

3.2.2. Ranked System Calls using Large Population Test (RSLPT)

Generally two sample large population test are used to determine equality in the means (μ) of populations. Motivated by this fact each system call in the trace is investigated, to check if the means (μ) are divergent in the target malware (M) and benign (B) classes. Hence, two tailed test are applied, subsequently hypothesis are defined as follows:

- Null Hypothesis: $H_0 : \mu_M^{s_i} = \mu_B^{s_i}$
- Alternative Hypothesis: $H_A : \mu_M^{s_i} \neq \mu_B^{s_i}$

Equations (8) through (12) are used to assign rank to a system call. As stated previously M and B represent malware and benign dataset. $\overline{x_B}$ and $\overline{x_M}$ are the mean of occurrences of system call in the benign (respectively malware) set. $\sigma_B^{s_i}$ and $\sigma_M^{s_i}$ is the standard deviation of a system call s_i with the occurrence x_j in the jth application. $StdErr(\overline{x_B} - \overline{x_M})$ depicts the standard error which is the sum of square of standard deviation ($\sigma_B^{s_i}$ and $\sigma_M^{s_i}$) divided by total number of benign (n_B) and malware files (n_M). $diff_{s_i}$ is the absolute difference of mean values of s_i in the target classes. ϕ_{s_i} is thus calculated which is the ratio of difference ($diff_{s_i}$) divided by standard error.

$$\sigma_B^{s_i} = \sqrt{\frac{\sum_{j=1}^{|B|}(x_j - \overline{x_B})^2}{n_B}} \tag{8}$$

$$\sigma_M^{s_i} = \sqrt{\frac{\sum_{j=1}^{|M|}(x_j - \overline{x_M})^2}{n_M}}, \tag{9}$$

$$StdErr(\overline{x_B} - \overline{x_M}) = \sqrt{\frac{\sigma_B{}^2}{n_B} + \frac{\sigma_M{}^2}{n_M}}, \tag{10}$$

$$diff_{s_i} = |\overline{x_{B,s_i}} - \overline{x_{M,s_i}}|, \tag{11}$$

$$\phi_{s_i} = \frac{diff}{StdErr(\overline{x_B} - \overline{x_M})}. \tag{12}$$

From the obtained ϕ value for each system call only calls having values greater than τ i.e., 1.96 (value corresponding to 95% confidence interval for large population test) are considered as they signify variability of means in both classes. Larger the value of statistics further is the distance/standard deviation of sample from the population mean. In particular, larger values of ϕ would depict separation of the vectors, thus improves classifier performance.

3.3. Construction of System call Occurrence Matrix

System call occurrence matrix is the collection of vectors consisting of N rows and $M+1$ columns refer Figure 3. Here each row of the occurrence matrix denotes a vector X of M dimensions obtained after pruning irrelevant attributes. The last column of each vector denotes the class labels (Y). In contrast to boolean values recorded in system call occurrence matrix that designate presence (1)/absence (0) of a system call in a sample. Each call is assigned a weight that expresses its importance in a particular application. Thus, the element (X_i^j) of system call occurrence matrix is TF–IDF weight of the jth system call. Generally, frequently invoked system calls get higher weight compared calls that are invoked by all samples in the dataset. In contrast TF–IDF incorporates frequent calls in an application (i.e local information) and also include infrequent/isolated calls from the collection of apps (global information) to compute the weight of a system call.

Using the TF–IDF values we represent each app as a vector. The component of an app vector is the TF–IDF weight of specific system call belonging to the attribute space. A classification algorithm is supplied with such call occurrence matrix for preparing the learning model. The classifier produces an appropriate result if there exists a substantial difference between vectors of target classes.

ACCEPTED MANUSCRIPT

Procedure 2 Significant System calls extracted using RSLPT

procedure RSLPT(S) ▷ $S = \{s_1, s_2, \cdots, s_{N_S}\}$, set of system calls.
  while $i \neq |S|$ **do**
    while $j \neq |M|$ **do**
      $sum_n \leftarrow sum_n + (x_j - \overline{x_{s_i}})^2$
    $\sigma_M^{s_i} = \sqrt{\frac{sum_n}{|M|}}$
    $sum_n \leftarrow 0$
    while $j \neq |B|$ **do**
      $sum_n \leftarrow sum_n + (x_j - \overline{x_{s_i}})^2$
    $\sigma_B^{s_i} = \sqrt{\frac{sum_n}{|B|}}$
    $diff_{s_i} = |\overline{x_{B,s_i}} - \overline{x_{M,s_i}}|$
    $Err_{s_i} = \sqrt{\frac{\sigma_B{}^2}{|B|} + \frac{\sigma_M{}^2}{|M|}}$
    $\phi_{s_i} = \frac{diff_{s_i}}{Err_{s_i}}$
    if $(\phi_{s_i} > \tau)$ **then**
      return s_i

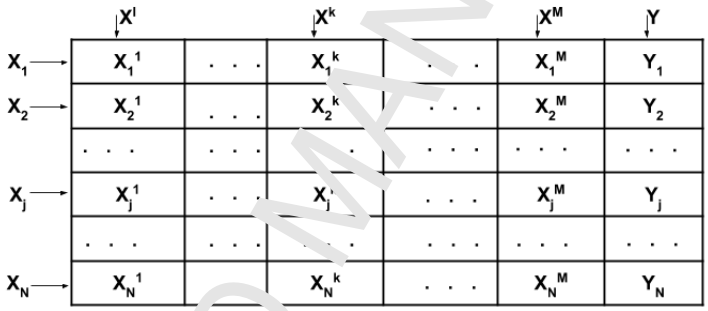


Figure 3: Feature occurrence matrix with instances, attributes and class labels.

3.4. Machine learning classifier phase

Multiple supervised learning classification algorithms such as Random forest, Rotation forest, AdaBoost (with J48 as base classifier) with the default setting in WEKA [54] is used. We employed 10-fold cross-validation [4], a statistical method for evaluating and comparing learning algorithms. In 10-fold cross-validation, entire dataset is divided into 10 subsets each of size $n/10$, with n considered as the size of the dataset. Performance of the classification model is estimated as average accuracy computed after 10 iterations on 9 training and 1 test sets.

3.5. Evaluation Parameters

Evaluation metrics such as false positive rate (*FPR*), accuracy (*A*) and Area Under ROC curve (*AUC*) are used as classifier performance indicators (refer Equations 13 and 14). Here, False positive (*FP*) indicates the misclassification of a benign app as malware. Truly classified benign files are indicated as true negative (*TN*). Malware samples correctly classified as malware are referred as true positive (*TP*). Malware instances classified as benign are known as false negative (*FN*). If a graph is plotted considering *FPR* values in abscissa and proportion of correctly classified malware instances in the ordinate the resulting curve is known as receiver operating characteristics (ROC) curve. The area in ROC curve corresponds to AUC. The AUC is between 0 and 1, the closer AUC value towards 1, the better the performance of classification model. So models with higher AUCs are preferred over those with lower AUCs. Thus, the detector is considered to be effective if *FPR* is minimum however large the values of *A* and *AUC* may be.

$$FPR = \frac{FP}{FP + TN} \tag{13}$$

ACCEPTED MANUSCRIPT

$$Accuracy(ACC) = \frac{TP+TN}{TP+FP+TN+FN} \tag{14}$$

4. Experiments and Results

The experiment was conducted on Ubuntu 14.04 platform with the support of Intel Core i7-4510U CPU @ 2.0GHz with 16 GB RAM. In order for an emulator to mimic real devices, we added an adequate number of contacts, periodically modifying battery status with random values and finally created the sufficient number of messages and call logs. To evaluate the consistency of our approach, experiments were initially performed on two datasets, later the scalability is estimated on three more datasets.

- Dataset1 (D1) contain 3130 benign applications downloaded from Google Play Store [57], Chinese market [58], Koodous [59], third-party Android markets [60] [61]. Each app was submitted to virustotal [53], an online scan engines to verify legitimate files were not injected with the malicious payload. A total of 2520 malware applications were randomly selected from Drebin dataset [5].
- Dataset2 (D2) contains 1514 malware samples downloaded from Koodous, user agencies and collection of ransomware from [56]. However, similar benign files considered in D1 constituted legitimate dataset.

Experiment were conducted in two different settings.

- Android Monkey [62] is used to generate random inputs. Initially, Android debug bridge (ADB), part of the Android SDK is used to install an APK into an emulator. The APK is launched and using strace [55] utility, and system calls are recorded. During experimentation 2500 random events were generated. When the interaction of Android monkey on an APK is completed the emulator process is killed and the clean AVD configuration is loaded.
- In the succeeding version of the experiment an experienced application developer also with the skill set in testing an app, interacted with it for a duration of 5 minutes. After the collection of logs, the system was restored in the clean state.

The following section explains the results obtained under different experimental setup.

4.1. Performance of Absolute Difference of Weighted System Calls

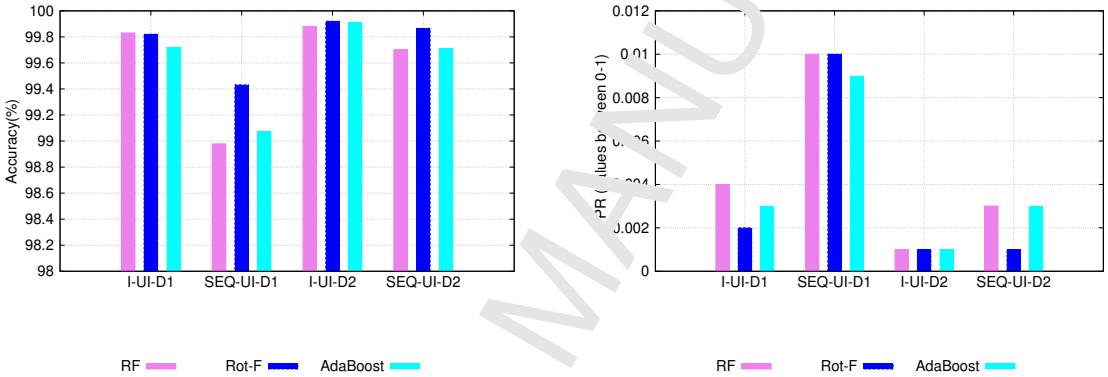
In this section, we determine the optimal feature length that resulted in an improved performance. We observe from Table 2 that Random forest provided an AUC value of 0.999 with a false positive rate of 0.001 at a feature length of 20, while Rotation forest resulted in AUC value 1.0 with 80 system calls. AdaBoost demonstrated an FPR of 0.002 and AUC of 0.998 with 10 features. Evaluation metrics depict that Random forest classifier outperformed other algorithms for samples in dataset 1. Similar trend in results are obtained for dataset 2. For dataset 2, an accuracy of 99.99% with 0.001 FPR is obtained with Rotation forest and AdaBoost for a collection of 10 system calls. Random forest classifier demonstrated an identical result with 20 calls.

Experiments with the similar setting is performed on the sequence of system calls. Two consecutive system calls are now considered as features. For D1, Random forest at feature length of 600 resulted in a low false positive rate of 0.006 and AUC value 0.99. Rotation forest depicted false positive rate of 0.005 and AUC 0.99 at feature length 700 and AdaBoost classifier showed a false positive rate of 0.003, AUC value 0.99. The sequence of system calls generated by Android Monkey for dataset 2 showed an identical pattern; with FPR in the range of 0.001 to 0.003, AUC in the range of 0.99 to 1.0 for feature length between 40-60. Using Figure 4, we show average accuracies and FPR for system calls considered independently and in sequence. We show that ADWSC extracted system calls on all classifiers exhibited accuracy exceeding 99.7%.

ACCEPTED MANUSCRIPT

Table 2: Results of ADWSC on independent system calls in D1 (UI Interaction)

FL	Random forest			Rotation forest			AdaBoost		
	ACC	FPR	AUC	ACC	FPR	AUC	ACC	FPR	AUC
10	99.81	0.002	0.99	99.44	0.006	0.99	99.81	0.002	0.99
20	99.91	0.001	0.99	99.72	0.003	0.99	99.63	0.004	0.99
30	99.81	0.002	0.99	99.81	0.002	0.99	99.63	0.004	0.99
40	99.91	0.001	0.99	99.81	0.001	0.99	99.63	0.004	0.99
50	99.91	0.001	0.99	99.91	0.001	0.99	99.81	0.002	0.99
60	99.91	0.001	0.99	99.81	0.002	0.99	99.72	0.003	0.99
70	99.81	0.001	0.99	99.91	0.001	0.99	99.63	0.004	0.99
80	99.81	0.002	0.99	100.0	0.00	1.00	99.81	0.002	0.99
85	99.57	0.025	0.99	100.0	0.00	1.00	99.81	0.002	0.99
avg.	**99.83%**	**0.004**	**0.99**	**99.82%**	**0.002**	**0.99**	**99.72%**	**0.003**	**0.99**



(a) Detection Accuracy

(b) False Positive Rate

Figure 4: Classifier performance for features selected with ADWSC. Independent and sequence of calls extracted with Android Monkey for D1 is denoted with I-UI-D1 and SEQ-UI-D1. Likewise, for D2 results are shown with I-UI-D2 and SEQ-UI-D2.

4.2. Evaluation of RSLPT pruned system calls

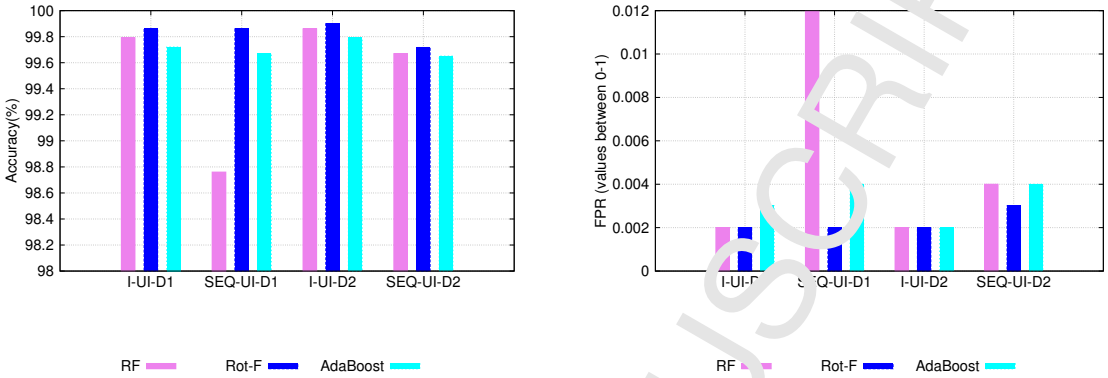
On examining the results obtained (see Table 3) for system calls mined with RSLPT, it can be concluded that performance with this feature selection approach is found to be similar to ADWSC. The evaluation was repeated by varying feature length considered in increments of 10 system calls. The overall performance of model obtained is, AUC of 1.0 with FPR between 0.001 to 0.002 at feature length in range of 10–30. For all classifiers, the similar trend in results is observed for D2. The sequence of system calls pruned with RSLPT, interacted with Android Monkey for D1 and D2 demonstrated an AUC of 1.0 at FPR in range of 0.005 to 0.007, on D1 and an AUC of 1.0 at variable FPR between 0.002 to 0.003 for D2 for all algorithms.

Table 3: Results of RSLPT on independent system calls in D1 (UI Interaction)

FL	Random forest			Rotation forest			AdaBoost		
	ACC	FPR	AUC	ACC	FPR	AUC	ACC	FPR	AUC
10	99.72	0.003	1.00	99.91	0.001	1.00	99.81	0.002	1.00
20	99.72	0.003	1.00	99.81	0.002	1.00	99.72	0.003	1.00
30	99.91	0.001	1.00	99.91	0.001	1.00	99.63	0.004	0.99
41	99.81	0.002	1.00	99.81	0.002	1.00	99.72	0.003	0.99
avg.	**99.79**	**0.002**	**1.00**	**99.86**	**0.002**	**1.00**	**99.72**	**0.003**	**0.99**

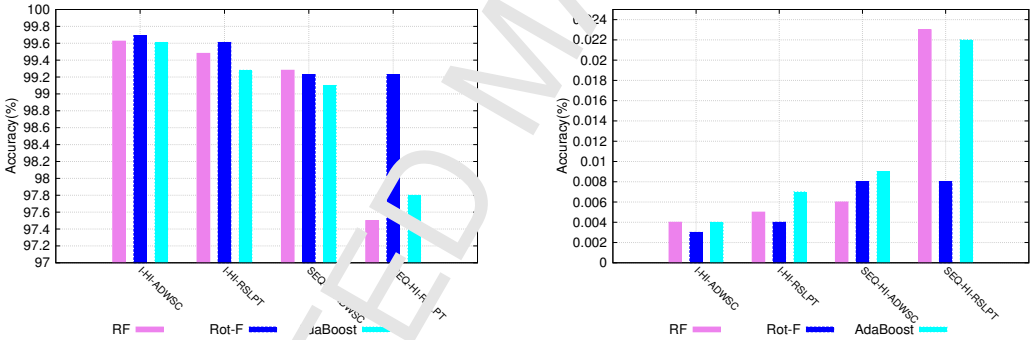
ACCEPTED MANUSCRIPT

Figure 5 shows average accuracy and FPR for different classification models. We can see that in majority of cases accuracies exceeds 99.6% and FPR does not surpass 0.004. In conclusion, experiments also presented the significance of ensemble classification algorithms for generating of system call-centric scanners.

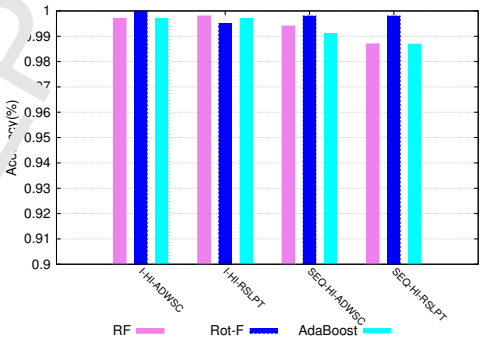


(a) Detection Accuracy (b) False Positive Rate

Figure 5: Classifier performance for features selected with RSLPT. Independent and sequence of calls extracted with Android Monkey for D1 is denoted with I-UI-D1 and SEQ-UI-D1. Likewise, for D2 results are shown with I-UI-D2 and SEQ-UI-D2.



(a) Detection Accuracy (b) False Positive Rate



(c) AUC

Figure 6: System call logged with human interaction and Classifier performance for features selected with ADWSC and RSLPT recorded with human interaction.

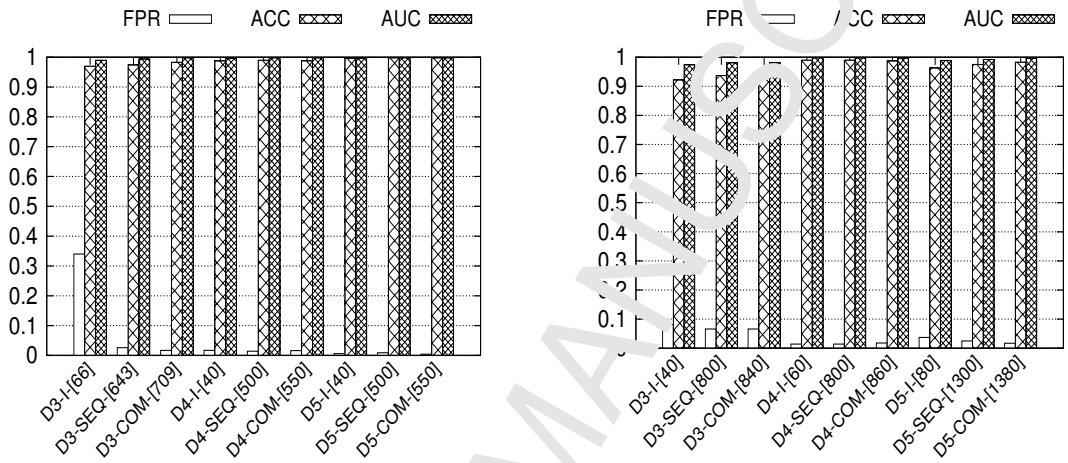
ACCEPTED MANUSCRIPT

4.3. Evaluation system calls logged with human interaction

For each event generated by a subject, set of invoked system calls are recorded. Figure 6 presents the average accuracy, FPR and AUC. Classification models developed with the proposed feature selectors and algorithms (Random forest and Rotation forest), depicted an accuracy higher than 99.4%. Besides, the AUC for aforementioned models is witnessed to be greater than 0.99 with FPR below 0.005.

4.4. Scalablity

Feature selection algorithms may produce promising results on specific dataset consisting of few hundred data objects. However, the performance can vary with samples from multiple sources with different number of examples.



(a) Performance metrics for ADWSC features

(b) Performance metrics for RSLPT features

Figure 7: Classifier performance for features selected with ADWSC and RSLPT.

To evaluate the scalability of proposed feature selectors, we gathered three dataset different from the samples used in previous examples which are discussed below.

- The third dataset (D3) is composed of 2474 malware samples from folders 1, 2 and 3 of Drebin [5] project, while same number of legitimate applications from Google Playstore are used.
- The fourth dataset (D4) is a collection of 2000 samples used by authors in [49]. Authors in their work stated that a total of 1000 malware samples were collected from four sources viz. Genome Project [50], Inter-Component Communication (IcRE) repository, New Malware Families 2015 and Contagio Minidump [63]. The trusted applications were randomly chosen from 12000 applications of Google Playstore.
- The Fifth dataset (D5) include 2412 applications. Out of these 1206 are malware and remaining 1206 are legitimate apps. To form malware dataset we collected samples from Genome Project [50] and benign files were downloaded from Google Playstore.

In Figure 7, we denote the X-axis labels as "Dx-I/SEQ/COM-[FL]". Here, Dx is the dataset name, I/SEQ/COM denote independent, sequence or combination of attributes, and FL is the feature length. In case of model developed with ADWSC feature selection algorithm on dataset D3 an accuracy of 98.3%, with 0.017 FPR and 0.998 AUC is obtained by combining independent and sequence of system calls. Identical trends are also obtained for datasets D4 and D5 respectively. For aforementioned datasets, highest accuracy in range of 98.3-99.8%, FPR between 0.004-0.017 and 0.999 AUC is achieved. On the contrary, models developed with RSLPT feature selector produced highest accuracy of 98.3%, with 0.017 FPR and 0.997 AUC for D5, moreover less accuracy i.e., 93.7% is obtained on dataset D3. The outcome of experiments demonstrate that model build using ADWSC attributes showed improved performance in comparison to models created using RSLPT.

ACCEPTED MANUSCRIPT

4.5. Performance of Classification Model formed using Privileged Applications and Malware

Malicious programs execute with root permissions to collect sensitive information from the devices/systems. In order to counteract malware infection, antimalware are utilized as a defense mechanism. A privileged application e.g. an anti-malware may use certain critical system calls to perform its functions. Since, the solution proposed by our approach is based on low level interaction (system calls) of the applications with the operating system, an identical distribution of the call sequence in malware and antivirus class may be expected to appear. In order to investigate the aforesaid presumption, we further collected 536 legitimate applications from Google Playstore consisting of antiviruses and system tools. Malware samples from Dataset D4 were utilized as it consisted of samples from diverse sources. Further, system calls were extracted in an identical manner as with the previous experiments. Subset of attributes were filtered using ADWSC feature selection method. We resorted on ADWSC opposed to RSLPT, as better results were obtained with the former approach in all the previous experiments. Since, the dataset is highly imbalanced, the performance of model was accessed with F-measure also including other metrics such as FPR and AUC as shown in Figure 8. An F-measure of 0.983 with 0.024 FPR and AUC value of 0.998 was obtained from model formed by combining 30 system calls and 300 sequence of attributes. The results of the experiments validate that the extracted features using ADWSC feature selection algorithm could appropriately differentiate samples of the aforementioned classes.

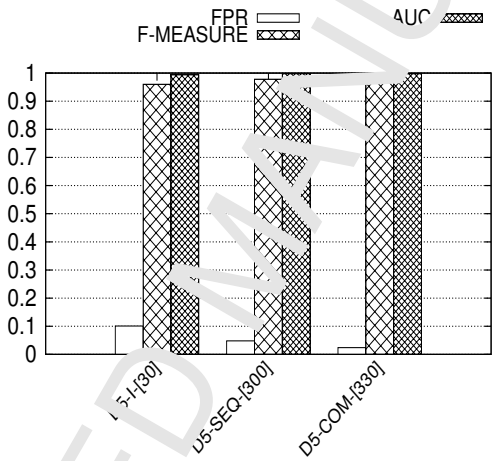


Figure 8: Performance metrics of classification model learned with attributes of malware and antivirus applications.

Table 4: Comparison of results on independent system calls

Datasets	Method	Classifier	FL	ACC	TPR	FPR	AUC
Dataset 1	ADWSC	Random forest	20	99.91	0.99	0.001	1.00
	RSLPT	Random forest	10	99.81	0.99	0.001	1.00
Dataset 2	ADWSC	Random forest	20	99.91	0.99	0.001	1.00
	RSLPT	Random forest	20	99.91	0.99	0.001	1.00
Dataset 3	ADWSC	Random forest	66	97.00	0.96	0.340	0.99
	RSLPT	Random forest	40	92.30	0.92	0.082	0.97
Dataset 4	ADWSC	Random forest	40	98.80	0.98	0.017	0.99
	RSLPT	Random forest	60	98.50	0.98	0.024	0.99
Dataset 5	ADWSC	Random forest	40	99.91	0.99	0.001	0.99
	RSLPT	Random forest	60	99.50	0.99	0.007	0.99

To determine if attribute set of antivirus and malware samples significantly differ, we computed zscore of each system call. We define null hypothesis, H_0, and alternative hypothesis, H_1 as:

H_0: malware and antivirus/system tool applications use the same set of system calls.

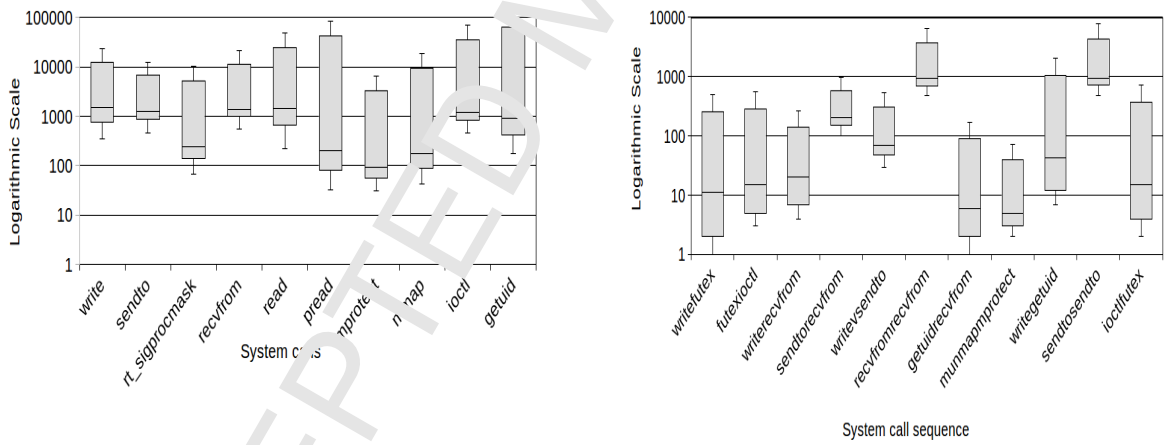
ACCEPTED MANUSCRIPT

Table 5: Evaluation of results on sequence of system calls.

Interaction	Method	Classifier	FL	ACC	TPR	FPR	AUC
Dataset 1	ADWSC	Random forest	600	99.44	0.99	0.0[illegible]	0.99
	RSLPT	Random forest	50	99.91	0.99	0.001	1.00
Dataset 2	ADWSC	Random forest	50	99.91	0.99	0.00[illegible]	1.00
	RSLPT	Random forest	300	99.72	0.99	0.003	1.00
Dataset 3	ADWSC	Random forest	600	97.60	0.97	0.026	0.99
	RSLPT	Random forest	400	93.60	0.9[illegible]	0.067	0.98
Dataset 4	ADWSC	Random forest	500	99.91	0.99	0.001	0.99
	RSLPT	Random forest	800	98.80	0.99	0.016	0.99
Dataset 5	ADWSC	Random forest	600	99.10	0.99	0.009	0.99
	RSLPT	Random forest	1300	97.50	0.97	0.027	0.99

H_1: malware and antivirus/system utilities use different set of system calls, thus the two classes statistically differ in the distribution of system call vectors.

After applying ztest, we obtained a set of 239 system calls having zscore greater than 1.96 (α=0.05), suggesting rejection of null hypothesis, H_0. Thus, we have strong evidence to accept alternate hypothesis, H_1, suggesting the two classes use different set of system calls. Further, we examined the system call distribution of prominent calls using box plot. Here, the box plot is used to validate if the distribution is normal. For normal distribution, the upper whiskers must be longer than Q_1 (first quartile) to the median. In Figure 9, X-axis shows the features and Y-axis is maximum value represented in logarithmic scale used for clearer visibility of top whiskers. The figures suggest that the large proportion of data is bounded towards the median, hence the distribution appears to be approximately normal.



(b)

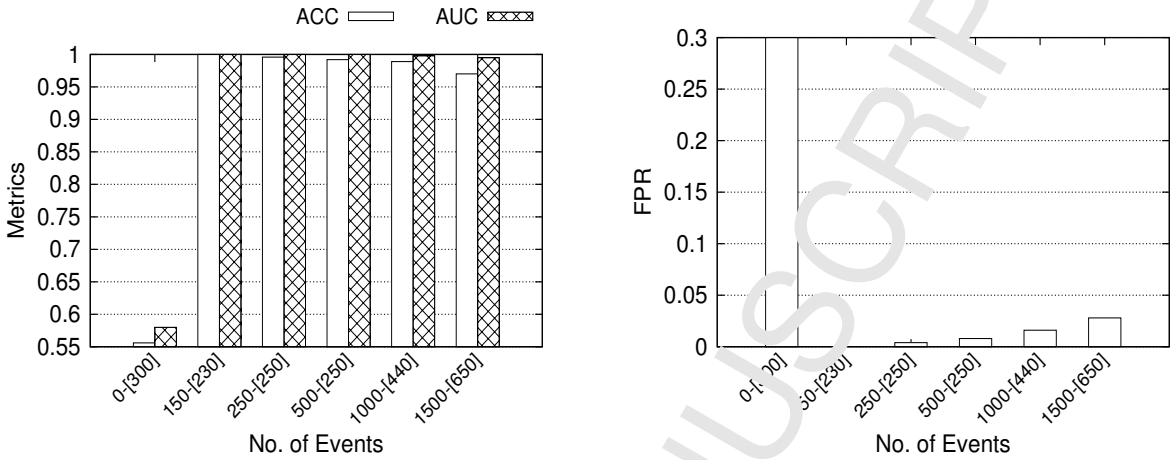
Figure 9: Box-plots for independent and sequence of system calls.

4.6. Evaluation based on Number of Events

In this experiments, we explore the classifier outcome on changing pseudo random events injected to Android Monkey during the application execution. We insert 150, 250, 500, 1000, and 1500 events. Subsequently, we inserted activities such as SMS text message, calling a phone number, altering orientation of screen, and location update into Android emulator. Additionally, for each pseudo random events, we used a constant seed value for all experiments. Figure 10 shows the quality of model. Classification model created with the feature set developed without random events exhibited poor results, indicating identical feature vectors of both classes. However, on increasing events,

ACCEPTED MANUSCRIPT

accuracy gradually improves and converges to a maximum at 150 events. Further, increase in the events added sparsity to the feature vector resulting in the drop of accuracy.



(a) Accuracy vs Area Under ROC

(b) False Positive Rate

Figure 10: Performance of classification model on varying pseudo random events. Labels in abscissa are denoted as *event-[feature-length]*.

4.7. Comparison of models constructed with ADWSC and RSLPT

Overall analysis of the experiments conducted in different settings on diverse datasets depicted accuracies exceeding 99.1% (see Table 4 and Table 5), except for D3. The applications in D3 where injected only with limited events like change of screen orientation and sending SMS messages. Also, experiments demonstrated that number of events are critical parameter influencing classification accuracy. As anticipated enhanced performances were obtained with features recorded for apps interacted with a subject. It was materialized as the subject supplied precise inputs for the appropriate functioning of the application. However, random inputs furnished by monkey runner sometimes transformed the app in a non-functional state. Hence, this led to an event where the accurate behaviour of the application is unexpressed. Optimal feature vectors were formed with the ADWSC approach. The attribute space solely consisted of system call characterizing substantial difference across the target classes. We also observed that set of system calls invoked by privileged (i.e., antivirus, system tools) and malware application exhibited difference in statistical distribution.

4.8. Comparative Analysis with Conventional Feature Selection Methods

Information Gain (IG) and Symmetric Uncertainty (SU) were traditionally used for synthesizing the attributes. Feature selection algorithms implemented in this paper is compared with IG and SU. Performance is estimated on variable values of threshold. In each settings threshold is halved and accuracy, FPR, and AUC is estimated (refer to Tables 6 and 7). Also, the average accuracy provided by IG and SU are 98.36% and 98.98% respectively. The output of experiments demonstrates the effectiveness of ADWSC and RSLPT over feature selection approaches such as IG and SU.

ACCEPTED MANUSCRIPT

Table 6: Performance with Symmetric Uncertainty Features(SU).

THRESHOLD	FL	ACC	FPR	AUC
0.500	2	94.283	0.063	0.955
0.250	8	99.114	0.011	0.996
0.125	20	98.792	0.015	0.997
0.063	30	99.114	0.013	0.998
0.031	37	98.953	0.014	0.998
0.016	37	98.953	0.014	0.998
0.008	41	98.872	0.015	0.999
0.004	41	98.872	0.015	0.999

Table 7: Performance with Information Gain Features(IG).

THRESHOLD	FL	ACC	FPR	AUC
0.500	6	97.984	0.027	0.992
0.250	11	98.792	0.015	0.996
0.125	27	99.194	0.011	0.998
0.063	33	99.033	0.011	1
0.031	37	99.194	0.011	0.998
0.016	40	99.194	0.011	0.999
0.008	41	98.953	0.015	0.999
0.004	44	100	0	1

Table 8: Performance with PCA at variable feature length. *max.FL* is the maximum feature length. Here, 25% features are obtained by dividing *max var* by 4; 50% features are obtained by dividing *max var* by 2 etc.

Metrics	**Permissions** *max var* = 2.604	**APIs** *max var* = 3.63	**System calls** *max var* = 3.198
max. FL	40	45	20
Accuracy	93.32%	95.1%	97.02%
FPR	0.088	0.052	0.043
AUC	0.971	0.988	0.995
max. FL	35	23	16
Accuracy[25%]	95.56%	95.34%	97.34%
FPR[25%]	0.075	0.050	0.039
AUC[25%]	0.982	0.989	0.995
max. FL	8	4	10
Accuracy[50%]	93.6%	94.32%	96.62%
FPR[50%]	0.083	0.048	0.043
AUC[50%]	0.981	0.974	0.993
max. FL	2	1	2
Accuracy[75%]	84.71%	76.06%	89.05%
FPR[75%]	0.177	0.261	0.119
AUC[75%]	0.922	0.793	0.921

Comparative analysis is also performed with Principal Component Analysis (PCA) [36], it is an example of dimensionality reduction approach. PCA examines the relationship amongst each variable to identify the ones which are more contributing. As it is a non-parametric method it provides a unique solution. However, the reduction of an irrelevant attribute might lead to loss of information. PCA assumes (a) dataset is a linear combination of the input variables (b) the direction of maximum variance guarantees the worthiness of features and (c) attributes having less variance are noisy. The features are also known as *principal component*, they are orthogonal to each other. Principal components which are orthogonal is informative compared to those having less variance.

PCA transforms the attribute space by changing the physical meaning of features. For permissions, APIs and system calls, we initially determine the principal component with maximum variance. Then, the performance of classification model is estimated using 25-75% of the variance corresponding to the top principal component (i.e principal component with the highest variance). On comparing three types of features, we again observe similar trends in the results i.e better performance is obtained with system calls, followed by permissions and API calls (refer Table 8).

ACCEPTED MANUSCRIPT

4.9. Evaluation of feature set

The relevance of feature set is determined by investigating the discriminative scores of each system call extracted with (a) ExtraTreeClassifier (b) selection of K best calls using `chi2` and (c) SelectKBest with `f_classif` score function, employing ANOVA F-value between label/feature. As the ranks assigned by the forementioned feature selector marginally differed, we show the outcome of ranked system call using ExtraTreeClassifier (see Figure 11).

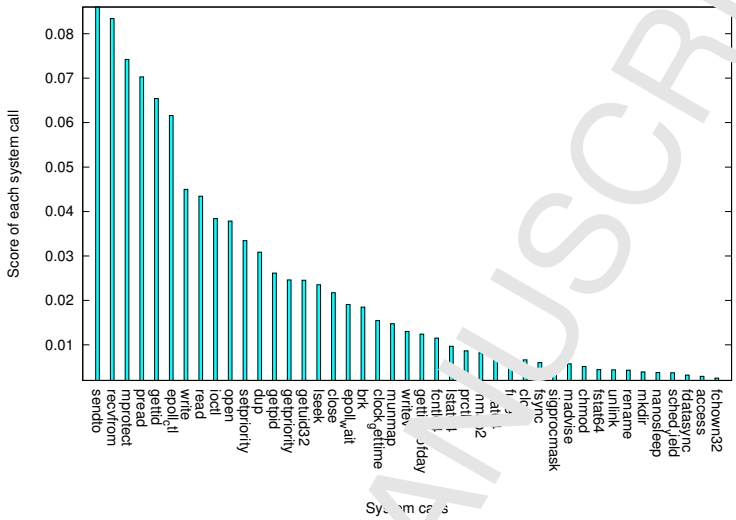


Figure 11: System call scores determined using ExtraTreeClassifier

An extra tree classifier implements meta estimator, which utilizes multiple decision trees on the subset of samples, to evaluate the importance of features. It creates extremely randomized classifier by choosing cut-points using the whole learning samples. Dropping the bootstrapped samples is an advantage in terms of bias, whereas the cut-point randomization minimizes the variance. SelectKBest chooses the top k system calls that have maximum relevance with the target classes. It takes two parameters as input arguments, 'k' and the score function to estimate the relevance of every attribute. The score function returns an array of scores for each attribute. Further, SelectKBest picks K system calls with highest scores. In this study, we have used `f_classif` and `chi2` as score functions. If we use `f_classif` then F-statistics for each attribute is calculated. If we employ `chi2` as a score function, the algorithm computes the `chi2` statistic between each system call and class labels. A small value indicates the call is independent of target classes. A large value denotes that the feature is likely to carry information about the classes. Additionally, we examined 20 prominent system calls ranked with above mentioned approaches, to ascertain if the set of significant calls are likely to be similar. The analysis assumes system calls ranked with ExtraTreeClassifier as the baseline, with which remaining methods are compared. 75%-85% system calls obtained with SelectKBest (`f_classif`) and SelectKBest (`chi2`) were found to identical to ExtraTreeClassifier. However, as the learning based approaches heavily rely on the distribution of data-points, this motivated us to further investigate the robustness of malware scanners on data and class label poisoning attacks, discussed in following sections.

5. Adversarial Attacks on System call based Scanner

The fundamental idea of any classifier is to estimate a boundary/decision surface that separates the training examples. An adversary can launch *poisoning attack* either have limited or complete knowledge of the system. By limited knowledge (i.e., the classification system is a black box) we mean, that the attacker is aware type of feature(system call), and the training set. On the contrary, in an complete knowledge attack, the adversary has full knowledge of target system. The knowledge of attacker includes the original training set, the mapping function of the features, and type of classifier and its parameters. An alternative form of attack is a *label flipping attack*. Here, the attacker forces training algorithms to learn the distribution of malware samples that are intentionally labeled as benign. In the following subsections, we briefly discuss data poisoning and label flipping attacks.

ACCEPTED MANUSCRIPT

5.1. Data poisoning attack

We assume that the attacker has complete knowledge of the classification system. This allows an adversary to create a model that is a replica of the target system being attacked. Furthermore, a similar attack can be extended if classification system is a black box, however, the adversary must have access to the classification scores. Thus, we implement data poisoning attack. In order to bypass anti-malware engines, hackers develop malware variants that implement code obfuscation techniques. Authors in [37] demonstrated an approach where variants of generated viruses could bypass commercial anti-viruses, however, was detected by Hidden Markov Model (HMM) detector. Subsequently, author in [38] created a virus generator with the ability to bypass both signature and HMM-based scanner. The morphed copies were generated using (a)equivalent code substitution (b)insertion of dead code using a block of instructions and subroutines from legitimate programs. Opcode mono-graph and di-graph frequency count having less than 30% similarity with the base malware thwarted the aforementioned detector. Motivated by these studies we implemented poisoning attack on system call based malware scanners. Ideally, we created evasive instances with the goal of maximizing misclassification rate.

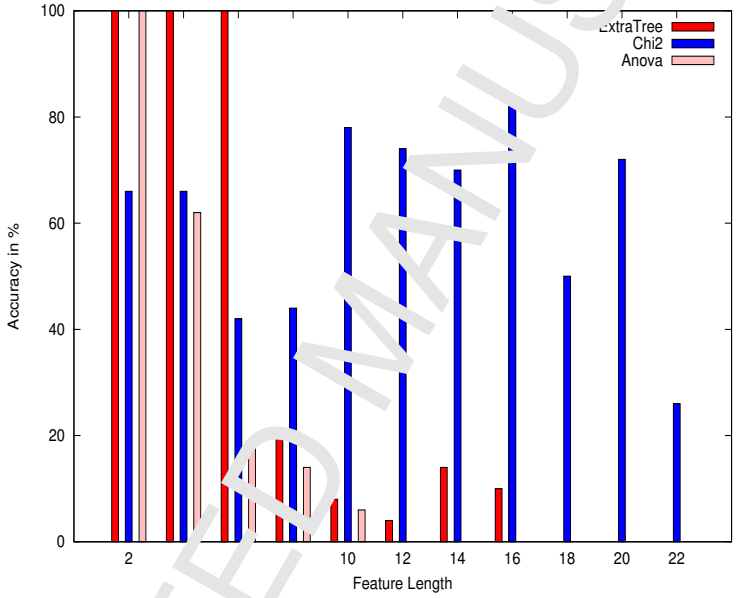


Figure 12: Performance of classifier on poisoning malware instances in order to transform them into evasive samples.

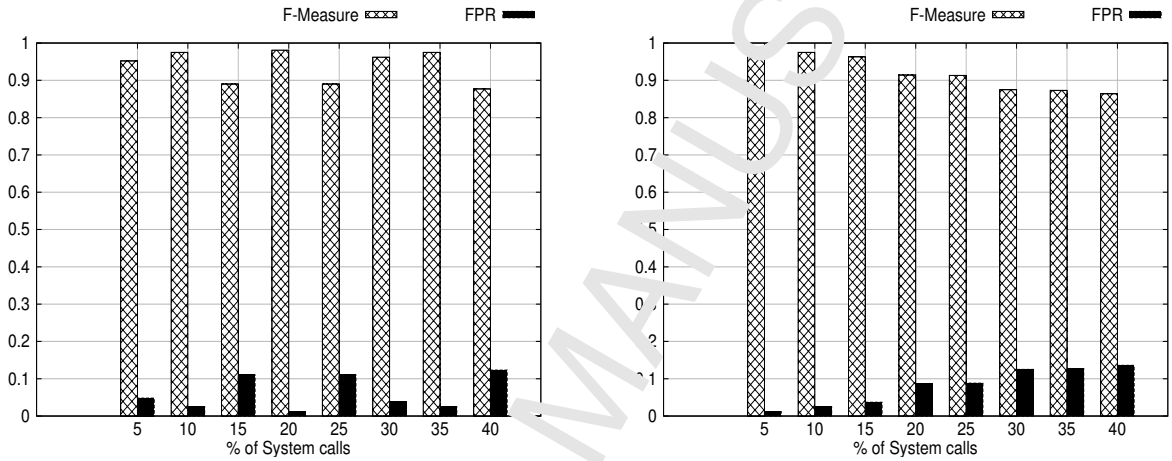
Ideally poisoning attacks are designed to maximize the loss function. Under such scenario, we mimic the ability of an adversary by fabricating copies of malware set, that approximate trusted samples. Towards this end, the maximum and minimum bound of the discriminative system call are determined. We run preliminary experiments on subset of malware samples. For this, 50 malware samples were randomly selected from dataset D1, and the occurrence of prominent system calls are modified with the maximum values in the benign instances. This attack appears to be realistic as the adversary can always access surrogate malicious and legitimate applications, which can be collected from publicly appearing repositories. We altered the variable number of system calls, obtained as an outcome of feature ranking methods discussed in Section 4.9. By modification, we intend to enhance the occurrence of system call(s) rather than decreasing it. As elimination or minimization of call(s) might lose maliciousness. Our focus was to transform malicious data points so that they statistically resemble legitimate feature vectors.

Malware samples were randomly selected, and values of discriminant system calls were modified. Two calls at each step were perturbed at a time until the classification accuracy drops (refer Figure 12). By performing this study, we also estimate the quality of feature ranking methods. If a larger number of attributes were required to be modified indicates robustness of the feature selector. Besides this, an adversary must invest substantial effort to fool detector. As a consequence there exists higher risk for an adversary being detected. Figure 12 summarizes the result of poisoning attack carried on 50 malware samples. We observe that system calls ranked with `ExtraTreeClassifier` leads to

ACCEPTED MANUSCRIPT

poor performance (i.e.,accuracy = 0%) on modifying 18 calls, `SelectKBest` with `Annova` as score function result in an accuracy of 0% by modifying 10 system calls. Further, `SelectKBest` with `Chi2` as score function shows a drop in accuracy (i.e., 28%) beyond 20 attributes, and remains constant until entire feature length.

For the sake of fair comparison, we extended our evaluation on dataset D4 containing malicious samples from multiple sources. We used 10% of malware samples for creating evasive examples, and reserved rest of the examples as training set. Two attack cases are implemented: (i) *bruteforce attack*: initially, 10% of benign samples are randomly selected from the training set. Later, randomly chosen attributes from these benign applications are interspersed in the subset (10%) of previously picked malware apps to form evasive instances and (ii) *specialized attack*: ranked attributes from training examples of benign set are derived, subsequently subset of malware samples are injected with these system calls. Evaluation are performed on two models, one formed using independent system call and other developed using sequence of calls. We report the results until 40% of attributes in the feature set are modified.



(a) Independent System Calls

(b) Sequence of System Calls

Figure 13: Classifier performance under bruteforce attack.

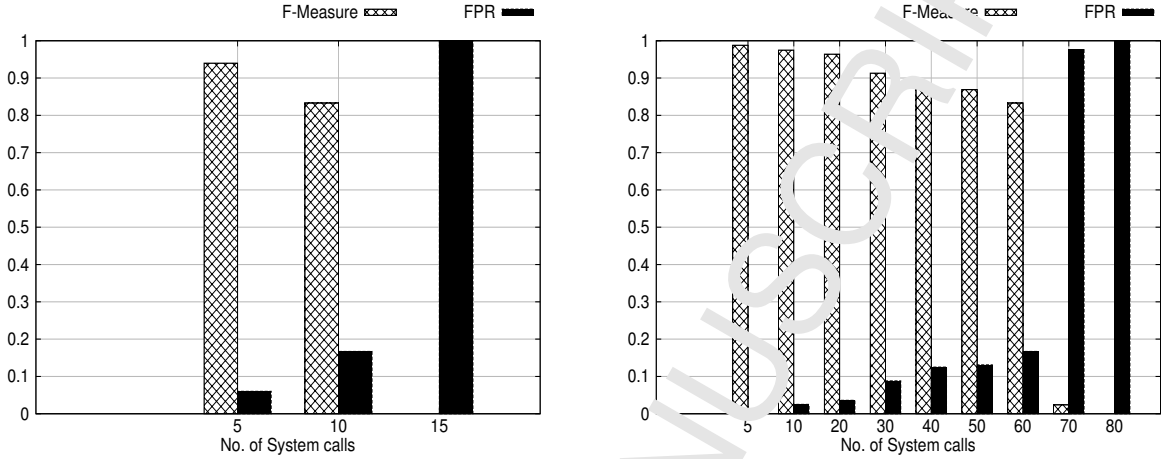
In the case of bruteforce attack, refer to Figure 13, lowest F-Measure of 0.877 with FPR value 0.123 is obtained for the classification model developed with independent system calls. Similarly, F-Measure drops to 0.860 with 0.136 FPR, for models created using sequence of system calls. It is worth noting that the bruteforce attack cannot substantially degrade the performance of classification algorithm. The underlying reason is that random attributes perturbed in the malware instances does not make the feature vectors similar to legitimate applications. An interesting insight in the behaviour of this attack reveals, modification of a larger number of attributes to produce the distribution of malware instances close to benign samples.

In the case of specialized attack, we performed a comprehensive analysis on the number of modified system calls to ascertain decrease in the classifier performance. Initially, we start modifying 5 attributes until 20% of features in the malware are altered. F-Measure drops to 0 on modifying 15 and 80 attributes respectively, reported in Figures 14a and 14b. This means that, in terms of attack sophistication, specialized attacks forces the classifier to mislabel more samples with less modified attributes compared to bruteforce attack.

We next present the result of inserting fraction of poisoned system calls on dataset D4. The amount of injected calls depend on the size of system call logs. For example, to add 10% of calls to a malicious instance consisting of 500 system calls, we inject 50 calls frequently accessed by legitimate applications. The results of our attack is presented in Figure 15, in which up to 50% of calls are augmented. From the results it can be clearly inferred that the classification degrades with the increase of injected system calls.

We conclude from exhaustive experiments, that traditional approach of building machine learning classifiers can break down against adversarial attacks. Thus, it is important for security analysts and developers to consider diverse aspects on how an adversary can adapt to the classification systems. This also raises a deeper concern on machine learning based security systems to be tested for effectiveness, before it can be deployed. In order, to overcome the

ACCEPTED MANUSCRIPT



(a) Independent System Calls

(b) Sequence of System Calls

Figure 14: Classifier perform[illegible]der specialized attack.

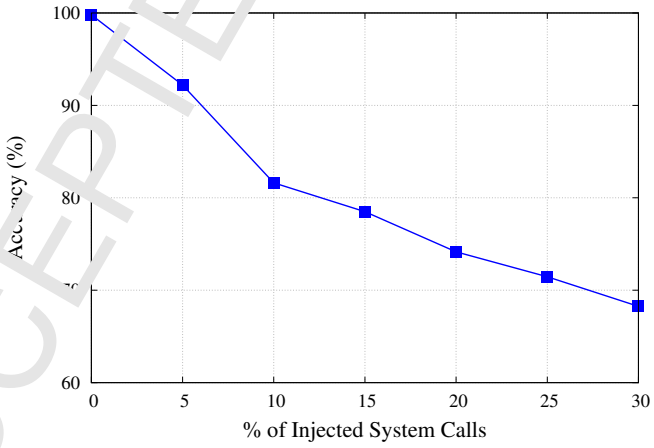


Figure 15: Classification accuracy as a function of the percentage of injected poisoned calls using data poisoning attack.

ACCEPTED MANUSCRIPT

attack, the training set is augmented with evasive samples. This generates a model which can match the distribution of data points in the training set. We analyze the outcome of feature selectors and determine the feature length for which accuracy drops to 0%. Later, we retrain our classification algorithms by including such examples. We observe that the performance of classification algorithm tremendously improves. Models are re-trained with previously used samples (i.e., 50 malicious instances), whose discriminant features where altered to match values with the trusted examples. Table 9 shows the outcome of the experiment.

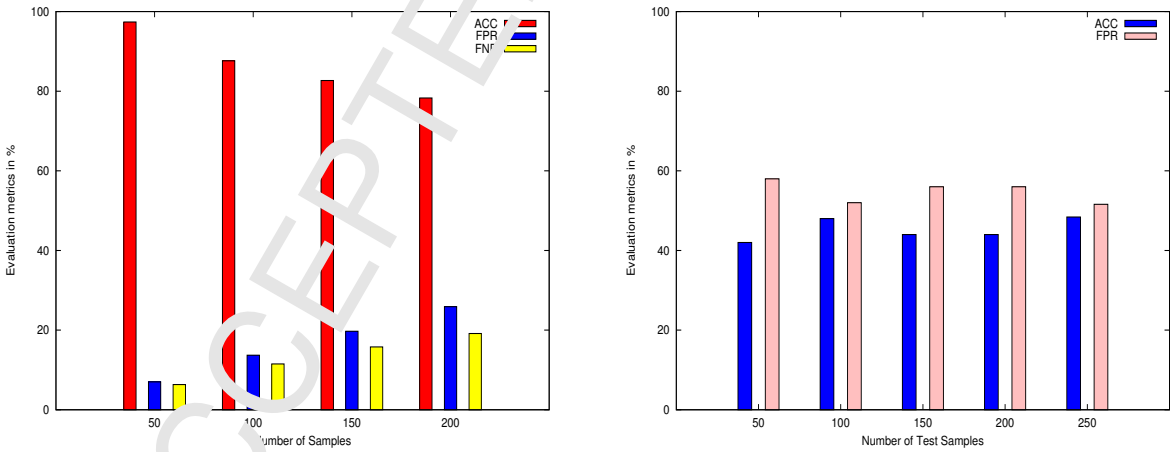
Table 9: Performance of Random forest retrained with evasive malware instances.

Feature Length	Accuracy	FPR
18 (ExtraTree)	99.7%	0.003
12 (Annova)	99.5%	0.003
>20 (Chi2)	99.76%	0.003

5.2. Label Flipping Attack

Label flipping attack is a carefully crafted attack, where the adversary is constrained by modifying labels of a subset of malicious training examples with the intention of minimizing classification error. From the practical viewpoint of the problem, there might arise a situation where end-user feedback is employed for labeling samples (e.g. collaborative spam filtering and online malware classification system). Thus, the users can intentionally mislabel the examples to create tainted data which is used for re-training the classifiers. Hence, the classifier decision boundary is updated with the distribution of contaminated samples, this can further affect the prediction.

We assume that adversary has complete knowledge of the system. The goal of the attacker is targeted to create a denial of service of attack i.e., to affect the availability of classification system. By this experiment, we aim to evaluate the effectiveness of Random forest with increasing fraction of flipped labels. In order to ascertain this task, we created malicious instances which are closer to the statistical distribution of malware instances in the training set. For each subset of samples randomly drawn from the training set, the performance is averaged over 5 iterations. Figure 16a reports the outcome of experiments. Here, we also report false negative rate (i.e., FNR which denotes the fraction of malicious samples incorrectly labeled as benign). The results indicate degraded performance on the of increase examples with flipped labels.



(a) Performance on subset of randomly selected samples, and average performance estimated over five iteration.

(b) Performance on synthetic malware samples.

Figure 16: Classifier performance obtained with label flipping attack.

In an alternative experimental setting, we evaluated the classification results by augmenting the training set with synthetic malware samples. Such samples were carefully created to have the statistical distribution which lies in

ACCEPTED MANUSCRIPT

the neighborhood of malicious samples in the training set but labeled as benign. In total we created 300 synthetic examples, likewise, the benign dataset was extended with the addition of 250 applications collected from playstore. The learning model is generated with the training set used in all previous experiments in addition to 50 samples extracted from the synthetic dataset, and remaining instances were reserved for testing. Figure 16b exhibits the result of classification. We observe that classification accuracy lie between 42-48.4%, with FPR in range of 51.6-58%. Extension to our work could be considering adversarial label noise attack where the attacker has limited knowledge i.e., when the feature set, training set or the classifier in unknown. This will help us investigate if the proposed attack can be transferred to limited knowledge case.

6. Discussions and Future Work

Exhaustive experiments on the dataset gave insight to some of the important findings. We observed that sequence of system call were effective in identifying malicious apps. In order to further show the effectiveness of system call-centric detector over scanners using static attributes, we created a similar set of the experiment after extracting subset of critical permissions and APIs. With Random forest algorithm learned with 70 permissions an accuracy of 92.37%, FPR value of 0.076 and AUC of 0.973 was obtained. Likewise, using 150 prominent APIs, accuracy and FPR were, 91.83% and 6.6% respectively. However, results furnished in Section 4, with attributes derived from the application of ADWSC, demonstrated an AUC of 0.99. List of a few significant system calls are shown in Table 10.

Table 10: Significant system calls pruned with feature selection approaches.

System call	Description
`sendto`	Transmit a message to another socket
`socketpair`	Create a pair of connected sockets
`recvfrom`	Receive a message from socket
`epoll_wait`	Wait for and I/O on file descriptor
`writev`	Writes data to file or socket
`dup`	Create a copy of file descriptor
`write`	Write to a file descriptor
`getpid`	Get process identification
`gettimeofday`	Get time and timezone
`readlink`	Places the contents of symbolic link in the buffer
`getuid32`	Returns user ID of the calling process
`clone`	Create a child process
`mprotect`	Set protection on region of memory
`ioctl`	Alter the device parameter of special files.

Normally, anti-malware using permissions can be deceived by hackers with malicious application developed using limited set of permissions, later the applications demand extra permissions at runtime. Additionally, sensitive APIs used to identify malicious applications can be defeated by using *reflection* APIs, which eventually installs malicious payload in the device during application update. Earlier studies [32] and [33] demonstrated a significant drop in antimalware detection rates due to the presence of obfuscated Android components. Also, combinations of diverse obfuscation techniques result in substantial change in the bytecode leading to failure in signature-based malware scanner. Methods such as string encryption, class encryption, app repackaging, junk code insertion and refinement of elements of resource folder might thwart scanners employing pattern/similarity-based approaches. However, malware scanners using system calls are relatively robust compared to static features mentioned before. Moreover, transformation could introduce sufficient degree of structural changes, perhaps cannot induce the way a program interact with an operating system and its related services using system calls. To hijack system call, attacker must first identify the address of system call table, then index at appropriate offset within the table to identify the address of the handler routine. Finally, she must overwrite address of handler with the address of malicious routine. However, attackers primarily face two obstacles (a) system call table information is not exposed by modern operating systems and (b) any memory update operation require both read-write permissions.

ACCEPTED MANUSCRIPT

Figure 17: Run time analysis of classification algorithms along with ADWSC

Through intensive experiments on five datasets, we noticed classifier performing poorly for feature set consisting of few attributes. However, as the features are progressively increased the performance improves. The primary reason is the variability across the samples enhances and classifications decreases. Further, augmenting system calls subvert original feature set. This results in lesser statistical diversity across feature vectors, yielding higher misclassification.

We also observed that the representation of elements of feature occurrence matrix influence the outcomes of classifiers. Normally, feature vectors are be represented with (a) boolean values which designate presence(1)/absence(0) of an attribute (b) frequencies of attributes or (c) associating weights to features. We investigated the influence of representing boolean and TF–IDF feature vector. Empirical evaluation depicted poor performance of classification model created with boolean feature vector compared to elements represented as TF–IDF values. Table 11 shows the results of constructing classification model, subject to input feature vector with elements as boolean values.

Table 11: Evaluation for boolean feature vector for datasets(D1) and dataset(D2) using system calls (S:denote independent and P: denote pair).✗denote apps interacted with human.

Feature Category	Feature Length	Android Monkey	A(%)	FPR	AUC
D1-S	10	✓	75.72	0.039	0.778
D2-S	10	✓	78.93	0.251	0.912
D1-P	200	✓	91.78	0.035	0.969
D2-P	50	✓	91.57	0.121	0.966
D1-S	20	✗	99.41	0.008	1
D2-P	100	✗	99.41	0.004	0.999

Further, we examined that Random forest and Rotation forest resulted in enhanced outcome, followed by AdaBoost (with J48 as base classifier). However, Rotation forest was found to be computationally expensive during model building phase. Thus, we preferred Random forest over Rotation forest (see Figures 17). In fact, it was observed from the study that all ensemble classification algorithms depicted marginal variation in the values of assessment metrics.

Additionally, we observed through extensive evaluation that machine learning based systems can be deceived by carefully modifying malicious applications. In particular, by changing values of 12-18 features, significant drop in

ACCEPTED MANUSCRIPT

the performance of the classification algorithms (refer to Figure 12) was noticed. Also, we show that poisoning the class labels of malicious instances results in higher misclassification rate (see Figure 16b). Retraining with adversarial examples improve detection. However, this also depends on the careful selection of the number of adversarial samples and classifier parameters. Otherwise, over-fitted model is created for model retrained with extremely large number of adversarial instances. Our approach suffers from a few shortcomings generally faced with malware scanner based on dynamic analysis these are discussed below:

- Usually dynamic analyzer produces limited code coverage of an application being instrumented. The functionality of an app is precisely exercised by subjecting it with sufficient events making use of automated input generators. However, these automated tools lack enough intelligence to appropriately determine app widgets and system events (with precise parameters). Moreover, such tools lack substantial knowledge of past events to generate new events causing it to explore incorrect path thereby resulting in abnormal termination of the application. Thus the development of input generation tool is an open research problem.

- Next challenge is to detect malicious samples that terminate execution on identifying the presence of emulator. Generally, such malware determines if emulator specific background process(s) exists in the process list. In order to avoid such evasive technique, the antimalware engines should simulate the process scheduling policies that are identical to the real device. Perhaps it would require emulator specific processes to be killed which would consequently bring down the performance of an emulator.

- As our approach is based on single app analysis, thus it would fail to identify pairs of applications that might independently appear legitimate, however, exhibit malicious behaviour if they collude with each other [35]. Colluding malicious applications can be detected on devices by initially listing entire applications, and generating a pair of calls that may leak sensitive information between source to sink applications. As the number of installed apps on the device are limited, a pair of calls which result in information leakage can be computed without substantial overhead.

- A specific type of privilege escalation attack is *confused deputy* attack [46], here malicious actions are performed by multiple apps rather than single app. Typically, such attacks may occur due to the presence of exported components, accessed by applications under the control of attackers. The identification of confused deputy attack may be dealt by in two stages. In the first stage, a coarse grained analysis might be conducted. Here, the analyzer identifies exported components, flag it insecure if the component is not protected with the required permissions. Later, fine grained analysis can be performed. In particular, control flow analysis may be done to identify exported components and the path to security sensitive methods in the flow graph. However, this approach comes with a limitation that the concrete classes cannot be identified due to inheritance and dynamic binding.

  Context switch between applications can be used to investigate confused deputy attack. This is ascertained by determining start of (a) activities (b) services or (c) receivers by Intents. Generally, Intents in Android can be specified in two ways (i) explicit and (ii) implicit. If suspicious applications use explicit Intents, then the target is assigned via constructor methods like setClass, setClassName and setComponent passed as parameter. For implicit Intents components are searched in global AndroidMainfest table. Appropriate components are located by backtracking the action, data field and eventually terminating when the action field in the Intent filter matches the one assigned in the Intent.

7. Comparative Analysis

In this section we provide a comparative evaluation of our approach with similar studies previously conducted. The summary of analysis is presented in Table 13. We also implemented similar feature selection methods reported by authors in [25] and [26] for synthesizing system call set derived using our dataset. As we had no access to dataset in [25] and [26], hence we experimented on the derived system call set from the collected samples. Refer to Table 12 for the outcome of results. We observe that the performance of malware detector employing absolute difference of system calls are inferior compared to feature selection approaches implemented in Section 3.2. Moreover, the fraction

ACCEPTED MANUSCRIPT

of relevant apps retrieved (i.e. recall) is significantly less compared to our methodologies (approx.99.8%). The principal limitation we observed with feature selection approaches implemented in papers [25] & [26] is the absence of class weight incorporated along with the frequency of system calls.

Table 12: Comparison of performance on our dataset while implementing the feature selection methods proposed in [25] & [26].

Metrics	TOP20	TOP30	TOP40	TOP50	TOP60	TOP66
Accuracy (%)	85.92	89	88.44	90.3	90.58	90.3
Recall (%)	80.9	86.9	87.1	88.8	89.2	89.6
ROC (%)	93.6	95.7	96.1	96.1	**96.5**	96

8. Conclusion

In this paper, the application of feature selection approaches and their performance was investigated. Evaluation was performed in two different experimental settings (a) using input generator and (b) employing human interaction. Impact of feature selection methods such as ADWSC and RSLPT on classifier performance was assessed. The assessment metrics for different settings of experiments exhibited an AUC in range of 0.99-1.0. To affirm the scalability of proposed machine learning approach evaluation was performed on five datasets. We also examined that enhanced outcome was reported by ADWSC feature selection approach.

We also investigated the robustness of the detector against adversarial examples. We report the modification of system calls in the range of 12-18 drops the performance of the detector to an extent that all malicious samples are labelled legitimate. We demonstrated an extension of the attacks by poisoning class labels, which also deceived the classifiers. Moreover, we show that retraining drops misclassification rates but at the cost of generating overfitted models. Demonstrated results demand the urgent need to develop secure malware scanners capable of recognizing adversary crafted inputs.

In future, we intend to conduct experiments on a larger dataset. It is also planned to consider attributes pertaining to network packets (packet size, size of payload in the packet, inter-arrival time of packets, the status of TCP flags, total number of bytes in packets, direction of packets, number of bytes sent/received etc.). The collection of these features bundled with suitable machine learning approach would certainly reveal promising patterns for identifying malware. It is also proposed to extend the present investigation to feature selection techniques that retain relevant attributes characterized by higher correlation with the class at the same time having lesser correlation with other features. Further, we plan to perform attacks on ensembles of classifiers.

References

[1] Breiman, Leo, 'Random forests', Machine Learning, 2001, pp.5-32

[2] Kuncheva, Ludmila I. and Rodríguez, Juan J., An Experimental Study on Rotation forest Ensembles, *Proceedings of the 7th International Conference on Multiple Classifier Systems*, 2007, pp.459-468

[3] Yoav Freund, Robert E. Schapire, A Short Introduction to Boosting, *Proceedings of the 16th International Joint Conference on Artificial Intelligence*, Volume 2, 1999, pp.1401-1406

[4] Payam Refaeilzadeh and Lei Tang and Huan Liu,Cross-Validation, *Encyclopedia of Database Systems (Springer US 2009)*, pp.532-538.

[5] Arp, Daniel and Spreitzenbarth, Michael and Gascon, Hugo and Rieck, Konrad and Siemens, CERT, DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket, *In the proceedings of 17th NDSS*, 2014, pp.181-196

[6] Hyo-Sik H, Mi-Jung C. Analysis of Android malware detection performance using machine learning classifiers. *International Conference on ICT Convergence (ICTC)*, 2013, pp. 490-495

[7] Yajin Zhou, Xuxian Jiang: Dissecting Android Malware: Characterization and Evolution, *IEEE Symposium on Security and Privacy*, 2012, pp. 95-109

[8] Andrea Gianazza and Federico Maggi and Aristide Fattori and Lorenzo Cavallaro and Stefano Zanero, PuppetDroid: A User-Centric UI Exerciser for Automatic Dynamic Analysis of Similar Android Applications, CoRR, 2014

[9] M. V. Varsha, P. Vinod & K. A. Dhanya: Identification of malicious Android app using manifest and opcode features, *Journal of Computer Virology and Hacking Techniques*, May 2016, pp. 1-14

[10] Gianluca Dini, Fabio Martinelli, Andrea Saracino, and Daniele Sgandurra: MADAM: a Multi-Level Anomaly Detector for Android Malware, *In proceeding of 6th International Conference on Mathematical Methods, Models and Architectures for Computer Network Security, MMM-ACNS*, 2012, pp. 240-253

ACCEPTED MANUSCRIPT

Table 13: Summary of comparative evaluation.

Approach	Features	Metrics	Remarks
[40]	135 permissions & 210 API	Precision & Recall in range of 98.31–98.91%	*Attacks on the anti-malware detector not discussed*. Permissions based detector can be easily misleading. Sophisticated malware request permission during runtime. Scanners employing APIs lead to increased false negative rate, as application might use reflection to call external functions. This behaviour is found extensively in legitimate applications.
[41]	permissions, sensitive API calls, system events and permission rate	accuracy is 88.26%	*Adversarial attacks not discussed*. Limitations existing in [40] is also applicable to this solution. Rotation forest needs more time during training phase.
[25]	*bigrams* of opcodes	accuracy is 96.88%	*Evaluation of adversarial attacks not performed.* Malware scanner developed with Opcode *ngram* can be defeated using naive approaches such as insertion of dead code, equivalent instruction substitution, instruction permutation, use of packers etc.
[39]	196 system calls	$F-score$ is 0.982, TPR is 0.977 & FPR is 0.013	*Attacks by an adversary, not addressed.* Malicious call sequence can be disturbed by injecting system calls found in trusted samples. Training is expensive.
[26]	system call sequences	accuracy is 97%	*Attacks on classification system not evaluated.* Limitations persisting in [39] is also applicable in this study. Feature selection method does not employ weight of class along with score assigned to system call.
[42]	system call sequences of variable depth	90% accuracy	*Adversarial attacks not addressed.* Patterns generated with a limited collection of malware & benign instances results in 7.69% FPR.
[49]	sequential system call graph (SCG) using Markov chain	94.2% accuracy	Rare and frequent system calls were inserted in malware sample. Experiments demonstrated resilience to system-call injection attacks.
our method	system calls	accuracy of 99.9% with AUC value 1.0	proposed feature selection methods, evaluated the performance with conventional filters and PCA. Evaluated the robustness of system calls against permissions and APIs. Empirically evaluated the robustness of classifier on class label and data poisoning attacks.

ACCEPTED MANUSCRIPT

[11] Ali Feizollah, Nor Badrul Anuar, Rosli Salleh, Ainuddin Wahid Abdul Wahab: A review on feature selection in mobile malware detection, *The International Journal of Digital Forensics & Incident Response*, Volume 13, June 2015, pp. 22-37

[12] Shabtai, Asaf and Kanonov, Uri and Elovici, Yuval and Glezer, Chanan and Weiss, Yae, Andromaly: A Behavioral Malware Detection Framework for Android Devices, *Journal of Intell. Inf. Syst.*, 2012, pp. 161-190

[13] L. Cen and C. S. Gates and L. Si and N. Li, A Probabilistic Discriminative Model for Android Malware Detection with Decompiled Source Code, *IEEE Trans. Dependable Sec. Comput.*, 2015, pp. 100-412

[14] Fairuz Amalina Narudin, Ali Feizollah, Nor Badrul Anuar: Evaluation of machine learning classifiers for mobile malware detection, *Journal of Soft Computing*, 2016, pp. 343-357

[15] Yerima, S.Y.;Sezer, S.;McWilliams, G.; Muttik, I., A New Android Malware Detection Approach using Bayesian Classification, *In Proceeding of 27 th IEEE International conference on Advanced Information Networking and Applications (AINA)*, 2013, pp.121-128, 25-28 March 2013

[16] Yerima, S.Y.;Sezer, and Muttik, I., Android Malware Detection using Parallel Machine Learning Classifier, *In 8th International Conference on Next Generation Mobile Apps, Services and Technologies*, Oxford, United Kingdom, pp 38–42, 2014

[17] Kuncheva, Ludmila I. and Rodríguez, Juan J., An Experimental Study on Rotation forest Ensembles, *In Proceedings of the 7th International Conference on Multiple Classifier Systems, MCS'07*, pp. 459–468

[18] M. S. Alam and S. T. Vuong, Random forest Classification for detecting Android Malware, *In Proc. of IEEE and Internet of Things (iThings/CPSCom), IEEE International Conference on and IEEE Cyber, Physical and Social Computing*, 2013, pp.663-669

[19] Hahn, Sebastian, Mykola Protsenko, and Tilo Mller, Comparative evaluation of machine learning-based malware detection on Android, *Sicherheit.*, pp 79–88, 2016.

[20] Su, Xin, M. Chuah, and Gang Tan., Smartphone dual defense protection framework: Detecting malicious applications in Android markets, *In Eighth IEEE International Conference on Mobile Ad-hoc and Sensor Networks (MSN)*, 2012

[21] Feldman, Stephen, Dillon Stadther, and Bing Wang, Manilyzer: automated Android malware detection through manifest analysis, *In Proc of 11th IEEE International Conference on Mobile Ad Hoc and Sensor Systems*, 2014

[22] Wolfe, Britton and Elish, Karim O. and Yao, Danfeng (Daphne), Comprehensive Behavior Profiling for Proactive Android Malware Detection, Information Security: 17th International Conference, ISC 2014, Hong Kong, China, October 12-14, 2014, Springer International Publishing, pp,328–344

[23] Aswini, A. M.; Vinod, P., Towards the Detection of Android Malware using Ensemble Features, *Journal of Information Assurance & Security*, 2015, Vol. 10 Issue 1, p9-21.

[24] Canfora, Gerardo and Mercaldo, Francesco and Visaggio, Corrado Aaron, A classifier of malicious Android applications, *In the Proceeding of 8th IEEE International Conference on Availability, Reliability and Security (ARES)*, 2013, pp 607–614.

[25] Gerardo Canfora and Andrea De Lorenzo and Eric Medvet and Francesco Mercaldo and Corrado Aaron Visaggio, Effectiveness of Opcode ngrams for Detection of Multi Family Android Malware 10th *International Conference on Availability, Reliability and Security*,2015.

[26] Canfora, Gerardo and Medvat, Eric and Mercaldo, Francesco and Visaggio, Corrado Aaron, Detecting Android Malware using Sequence of System Calls, *In Proceedings of 3rd International Workshop on Software Development Lifecucle for Mobile*, 2015, pp 13–20.

[27] Grosse K., Papernot N., Manoharan P., Backes M., McDaniel P., Adversarial Examples for Malware Detection. In: Foley S., Gollmann D., Snekkenes E. (eds) Computer Security-ESORICS, Lecture Notes in Computer Science, vol 10493, Springer, Cham.

[28] H.H. Kim and M.J.Choi, Linux Kernal–based feature selection for Android Malware Detection, *The 16th Asia-Pacific Network Operations and Management Symposium*, 2014, pp.1–4.

[29] Lena Chekina and Dudu Mimran and Lior Rokach and Yuval Elovici and Bracha Shapira, Detection of deviations in mobile application network behaviour, arXiv preprint arXiv:1208.0564(2012)

[30] Chandrashekar, G. and Sahin, F., A survey on feature selection methods, *In Journal of Computers & Electrical Engineering*, 40(1), pp.16-28, 2014.

[31] Lin, Ying-Dar and Lai, Yuan-Cheng and Lu, Chien-Nan and Hsu, Peng-Kai and Lee, Chia-Yin, Three-phase behavior-based detection and classification of known and unknown malware, *In Security and Communication Networks*, vol. 8, pp 2004–2015, 2015.

[32] Jonghwa Park and Hyojung Kim and Younsik Jeong and Seong-je Cho and Sangchul Han and Minkyu Park, Effects of Code Obfuscation on Android App Similarity Analysis, *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications (JoWUA)*, vol. 6, no. 4, pp 86–98, December 2015

[33] V. Rastogi, Y. Chen and X. Jiang, "Catch Me If You Can: Evaluating Android Anti-Malware Against Transformation Attacks," in *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 1, pp. 99-108, Jan. 2014.

[34] Parampalli, Chetan and Sekar, R. and Johnson, Rob, A Practical Mimicry Attack Against Powerful System-call Monitors, *In Proceedings of the 2008 ACM Symposium on Information, Computer and Communications Security*, ASIACCS '08, 2008, pp.156–167.

[35] Shweta Bhandari and Vijay Laxmi and Akka Zemmari and Manoj Singh Gaur, Intersection Automata Based Model for Android Application Collusion,In the Proceedings of 30th IEEE International Conference on Advanced Information Networking and Applications, AINA 2016, Crans-Montana, Switzerland, 23-25 March, pp. 901–908, 2016.

[36] Jonathon Shlens, A Tutorial on Principal Component Analysis, *In Systems Neurobiology Laboratory, Salk Institute for Biological Studies*, 2005.

[37] Wong W., Stamp M., Hunting for metamorphic engines. *J. Comput. Virology*, 2(3), 211–229, 2006.

[38] Madenur Sridharan, Sudarshan and Stamp, Mark, Metamorphic worm that carries its own morphing engine, *In Journal of Computer Virology and Hacking Techniques*,2013,vol.9, no.2,pp 49–58.

[39] X. Xiao, Z. Wang, Q. Li, S. Xia and Y. Jiang, "Back-propagation neural network on Markov chains from system call sequences: a new approach for detecting Android malware with system call sequences",*In IET Information Security*, vol. 11, no. 1, pp. 8-15, 1 2017.

[40] Shina Sheen, R. Anitha, V. Natarajan, "Android based malware detection using a multifeature collaborative decision fusion approach", *In Neurocomputing*, Volume 151, Part 2, 2015, pp.905-912.

[41] Hui-Juan Zhu, Zhu-Hong You, Ze-Xuan Zhu, Wei-Lei Shi, Xing Chen, Li Cheng, "DroidDet: Effective and robust detection of android malware using static analysis along with rotation forest model", *In Neurocomputing*, Vol. 272, 2018, pp.638-646.

[42] Fei Tong and Zheng Yan, A hybrid approach of mobile malware detection in Android, *In Journal of Parallel and Distributed Computing*,

ACCEPTED MANUSCRIPT

vol(103), pp.22-31, 2017.

[43] Abdelfattah Amamra, Jean-Marc Robert, Andrien Abraham, Chamseddine Talhi, Generative versus discriminative classifiers for android anomaly-based detection system using system calls filtering and abstraction process, *In Security and Communication Networks*, 2016, 9, 16, 3483.

[44] Wei Wang and Yuanyuan Li and Xing Wang and Jiqiang Liu and Xiangliang Zhang, Detecting Android malicious apps and categorizing benign apps with ensemble of classifiers, *In Future Generation Computer Systems*,vol.78, pp.987-994, 2018.

[45] Milosevic, N., Dehghantanha, A. and Choo, K.K.R., Machine learning aided android malware classification. *In Journal of Computers & Electrical Engineering*, 61, pp.266-274, 2017.

[46] Hardy, N., 1988. The Confused Deputy:(or why capabilities might have been invented). ACM SIGOPS Operating Systems Review, 22(4), pp.36-38.

[47] Idrees, F., Rajarajan, M., Conti, M., Chen, T.M. and Rahulamathavan, Y., 2017. PIndroid: A novel Android malware detection system using ensemble learning methods. *Computers & Security*, 68, pp.36-46.

[48] Arshad, S., Shah, M.A., Wahid, A., Mehmood, A., Song, H. and Yu, H., 2018. SAMADroid: A Novel 3-Level Hybrid Malware Detection Model for Android Operating System. *IEEE Access*, 6, pp.4321-4339.

[49] Bhandari, Shweta and Panihar, Rekha and Naval, Smita and Laxmi, Vijay and Zemmari, Akka and Gaur, Manoj Singh, SWORD: Semantic aWare andrOid malwaRe Detector, *In Journal of Information Security and Applications*, 42, pp.[illegible], 2018.

[50] W.Zhou, Y.Zhou, X.Jiang, and P.Ning, "Detecting repackaged smart-phone applications in third-party android marketplaces", in CODASSPY, 2012.

[51] "Appbrain", `http://www.appbrain.com/stats/number-of-Android-apps`

[52] "Apktool", `https://ibotpeaches.github.io/Apktool/`

[53] "Virustotal", `https://www.virustotal.com`

[54] "WEKA", `http://www.cs.waikato.ac.nz/ml/weka/`

[55] "strace", `http://linux.die.net/man/1/strace`

[56] "Ransomware", `http://ransom.mobi/`

[57] "Google play", `https://play.google.com/store?hl=en`

[58] "Baidu", `http://shouji.baidu.com/`

[59] "Koodous", `https://koodous.com/`

[60] "1Mobile Market", `http://m.1mobile.com/me.onemobile.android.html`

[61] "9apps", `http://www.9apps.com/`

[62] "Android Monkey", `https://developer.Android.com/studio/test/monkey.html`

[63] "Contagio mobile malware mini dump", `http://contagiominidump.blogspot.com`

ACCEPTED MANUSCRIPT

Vinod P., is Post Doc at Department of Mathematics, University of Padua, Italy. Prior to joining Padua he was working as Professor in SCMS School of Engineering & Technology, Cochin, India. He holds his Ph.D in Computer Engineering from Malaviya National Institute of Technology, Jaipur, India. He has more than 50 research articles published in peer reviwed Journals and International Conferences. He is reviewer of number of security journals, and is also serving as programme committe memeber in the International Conferences related to Computer and Information Security. Vinod's area of interest is Adversarial Machine Learning, Malware Analysis, Context aware privacy perseving Data Mining, Ethical Hacking and Natural Language Processing.

Dr. Akka Zemmari has received his Ph.D. degree from the University of Bordeaux, France, in 2000. He is an Associate Professor in computer science since 2001 at University of Bordeaux, France. He has more than 100 research articles published in peer reviwed Journals and International Conferences, and also serves as programme committe memeber in the International Conferences. His research interests include distributed algorithms and systems, graphs, randomized algorithms, machine learning and security.

Mauro Conti (SM'14) received the Ph.D. degree from the Sapienza University of Rome, Italy, in 2009. He is currently an Associate Professor at the University of Padua, Italy. After his Ph.D., he was a Postdoctoral Researcher with Vrije Universiteit Amsterdam, The Netherlands. In 2011, he joined the University of Padua, where he became an Associate Professor in 2015. He was a Visiting Researcher with GMU (2008), UCLA (2010), UCI (2012, 2013, and 2014), and TU Darmstadt (2013). He received a Marie Curie Fellowship (2012) from the European Commission, and a Fellowship from the German DAAD (2013). His main research interest is in the area of security and privacy. In this area, he has authored over 100 papers in topmost international peer-reviewed journals and conference. He is an Associate Editor of several journals, including the IEEE COMMUNICATIONS SURVEYS AND TUTORIALS. He was the Program Chair of TRUST 2015, and the General Chair of Secure Comm 2012 and ACM SACMAT 2013.

ACCEPTED MANUSCRIPT

[1] We evaluated the robustness of system call based machine learning malware detector by comparing it with traditional features such as permissions and APIs.
[2] We proposed and implemented two feature selection approach and compared with the conventional algorithms like information gain, symmetric uncertainity and PCA.
[3] We proposed and implemented adversarial attacks i.e., label flipping and data poisoning attacks to evaluate the if system call based detector are secure and can withstand adversarial attacks.
[4] Also, we evauated the performance on adversarial countermeasures.
[5] Extensive experiments were conducted to determine optimal feature vector over benchmark data set.

ACCEPTED MANUSCRIPT